CAMPUS KOALA - SCAFFOLD

Link to GitHub - https://github.com/Jake-Handel/Campus-Koala

Jake Handel - 39900467

Table of Contents

Section 1 - Identifying and Defining (15 Marks) ***2***

1.1 — Client Scenario **2**

1.2 — Environmental Specification **4**

1.3 — List of Objectives **6**

1.4 — Performance Criteria **8**

1.5 — IPO (Input-Process-Output) **9**

1.6 — Gantt Chart **11**

Milestone 1: This should be completed by term 1, week 2. 11

Section 2 - Research and Planning (10 Marks) ***12***

2.1 — Research **12**

Market Study 12

User Research 13

Competitor Analysis 13

Summary of Findings 15

2.2 — Data Flow Diagram (DFD) **16**

2.3 — Interface Design (Wireframe) **19**

2.4 — Storyboard **20**

2.5 — Pseudocode **24**

Milestone 2: This should be completed by term 1, week 5. 28

Section 3.1 - Development (10 Marks) ***29***

3.1.2 — Logbook **29**

Milestone 3.1: This will be checked at various points each term. 40

Section 4 - Testing and Maintaining (10 Marks) ***41***

4.1 — Acceptance Testing **41**

4.2 — Load Testing **46**

4.3 — User Survey **50**

Section 1 - Identifying and Defining (15 Marks)

1.1 — Client Scenario

Define a clear scenario based on the client's needs, including the application's purpose, target audience, and primary goals. Detail the client's requirements and provide a narrative that encapsulates the problem that the application aims to solve. Explain the overall vision and the expected impact of the application.

Application purpose:
This application is an organisation assistant. It aims to assist students in guiding their study and assignments through school. It has a dual purpose of enabling students to plan out their school requirements and can be used by others such as adults to plan out their day effectively. The purpose of the web application is to allow people to be more organised using a to-do list, calendar, day planner, study timers with a built in Artificial intelligence (AI) chat bot to assist in any questions.

Primary Goals:

1. Secure and simple login and register pages
2. Easy to use user interface (UI)
3. To-do lists integrated with a calander
4. Day planning with study timers built in
5. AI integration to assist in study

Target Audience:
The target audience is school students and university students who often need assistance in organising their daily activities. Although it is mainly targeted towards school students, adults can also use it to plan their day, organise their tasks and plan important dates in the calendar. This can be particularly useful for families in juggling multiple commitments such as sports, school, and medical appointments.

Client requirements:
User registration and profile management:
Allow for users to register and see their profile. Securely store and manage users' information and details in their database.
To-do List:
Make, delete, complete, and edit different to-do. Synchronise with the calendar.
Calendar:
Make, delete, and edit different components in a calendar. Derive a to-do list with a date.

Day Planner:
Allow people to accurately plan their day and edit their own start and end while adding breaks when needed.

Study timers:
Set users own timers then get 1/3 of that for an integrated game time to relax the mind before another study timer.
AI Assistant:
Available for the users to ask an AI assistant with text prompts on any page after logging in.

Problem Narrative:
For the majority of students (and to a lesser extent adults) struggle with organising their time effectively, hence leading to decreased productivity, missing deadlines, and increased stress. The solution for task management, scheduling, and time tracking are quite often disjointed and lack connectivity, making it harder for users to maintain a cohesive and efficient work ethic. Readily available applications can be expensive and/or disjointed from a functionality perspective.

Overall, Vision:
The vision of the application is to transform time and task management for students and professionals with a comprehensive, integrated application that combines a to-do list, calendar, day planner, and study timer functionalities. The vision is to create a seamless cost effective, user-friendly platform enhancing productivity, reducing stress and fostering efficient work and study habits.

By offering a unified tool, the app will simplify task organisation and scheduling, streamline workflows, and help users maintain focus whilst incorporating a type of reward system where better study behaviours result in scheduled down time for fun activities. Customisable features and a supportive community will provide personalized experiences and social motivation.

1.2 — Environmental Specification

Outline the technical environment in which the application will operate. This includes hardware, software, network specifications, and other relevant environmental factors. Describe the operating systems, development tools, servers, and other infrastructure components necessary for the application.

	Supported Operating Systems	Hardware Requirements	Network Connectivity	Additional Notes
Google Chrome	- Chrome OS, Windows 10 or later, macOS, Linux (latest versions) - Can also use ChromeOS Flex from a USB	- Architecture: Intel or AMD x86-64-bit compatible processor - Processor: Intel Core i3 (or equivalent) or better - RAM: 4 GB minimum - Storage: 16 GB SSD or higher Graphics: Intel integrated graphics (newer than 2010). Note: Devices with Intel GMA 500, 600, 3600, and 3650 graphics do not meet performance standards.	Minimum download speed of 10 Mbps for seamless browsing and cloud functionality. A stable internet connection via ethernet or WIFI is essential.	Older hardware may lead to degraded performance, particularly devices manufactured before 2010.
Mozilla Firefox	- Windows: Windows 10 or later - MacOS: Version 10.15 (Catalina) or later - Linux: Most recent distributions	Windows: - Processor: 1 GHz or faster - RAM: 1 GB minimum - Storage: 500 MB free space macOS: - Processor: Intel x86 or Apple Silicon - RAM: 512 MB minimum - Storage: 200 MB free space	Minimum download speed of 5 Mbps for seamless browsing and cloud functionality. A stable internet connection via ethernet or WIFI is essential.	N/A

General System Requirements

- **Internet Connection:**
  - A stable internet connection is essential for application functionality. Minimum speed of 5-10 Mbps is recommended for optimal performance.
- **Browser Environment:**
  - Ensure you are using the latest stable version of **Google Chrome** or **Mozilla Firefox** for optimal compatibility, security, and performance.
  - Regular browser updates are encouraged to maintain security and stability.

Development Tools & Infrastructure

1. Operating Systems for Development:

- Development will be conducted on **Windows 10/11**, and **macOS** to ensure cross-platform compatibility.

2. Backend Infrastructure:

- **Backend Technology:**
  - **Node.js** or **Python** (with Flask) will be used to manage the server-side logic and APIs.

3. Database Servers:

- MySQL for structured data storage that is relational for users to hold multiple stacks of data.
- Deploying later and with a wider audience, cloud storage with Amazon Web Services or Google's cloud platform for stable and secure storage

5. Development Tools & Frameworks:

- **Frontend Framework:**
  - **React.js** for building dynamic, interactive user interfaces.
- **Version Control:**
  - **Git**, with repositories hosted on **GitHub**, to manage code versions and collaboration.

1.3 — List of Objectives

Enumerate the specific objectives that the application aims to achieve. These should be clear, measurable goals that directly address the client's needs. The objectives could include functionality, user engagement, performance targets, and other relevant criteria.

Functional Features

Feature	Priority	Description	Detail	Measurability
Login and register	High Priority	The user can login and register	Functional and secure register and login page for users, producing a token at login.	Within 1-2 seconds of handling time
Tasks	High Priority	The user can add, edit, delete and complete tasks	Users can make tasks and then edit, complete or delete them from their view.	Within 1 second handling time.
Calendar	High Priority	Create, edit, and delete calendar events	Users can make, edit and delete calendar events.	Within 1 second of handling.
Synchronise the tasks to calendar	Medium Priority	A task from the to-do list with a date will sync with the calendar app	The task from the Todo list page with a date attached to it, will automatically sync to the calendar.	Within 1 second of handling.
Day Planner	High Priority	Adjust and edit their day planner to a user's need	Users can schedule and set planned sessions to organise their day up.	Less than 1 second handling.
Study Timers	High priority	Start the day planner or adjust a personal timer for the user's preference	The study timer should be rendered locally hence very minimal latency, and a timer accurate to the time set is displayed and counted down.	Less than 1 second handling.
Break Timers	Medium Priority	Once the study timer goes off the user will have 1/3 of their study time allocated to have a break	Break timers are accurate and rendered and calculated locally hence low latency.	Accurate to 1/3 of the study time.
Games	Medium Priority	In the break, users can play simple but engaging games	Games can be chosen from Snake, Tetris, and Minesweeper. Once the break timer is done the game popup will close.	Games should be bug free with score handling and high response.
AI Assistant	Medium Priority	An AI assistant is in the bottom right of each page in which a user can use as needed.	Users can access the AI assistant on each page.	AI assistant loaded within 3 seconds due to the external API. The response should be filtered and displayed correctly.
Dark Mode	Low Priority	Switch from light and dark mode for the user's preference.	Dark mode should have a consistent theme, contrasting from light mode and holding the same sort of theme throughout.	Dark mode consistent throughout.

Non-Functional Features

Feature	Priority	Description	Measurability
Engagement	Medium Priority	Consistent usage from users, and recommendation to peers.	Users Preference and survey feedback.
Performance	High Priority	Smooth functionality, quick interface, no bugs.	Average below 1.5 seconds to handle and load requests.
Looks	Low Priority	Engaging colour scheme. Looks modern and user friendly, while looking professional.	Users Preference and survey feedback.
Game Like	Medium Priority	Make studying more like a game with targets to achieve, making it more engaging and fun for users.	Summarised numbers and key points that pop out to the user. Users Preference and survey feedback.

1.4 — Performance Criteria

Specify the application's performance criteria, including speed, reliability, scalability, and user satisfaction benchmarks. Define how performance will be measured and the application's standards.

The focus is on delivering a high-quality user experience by ensuring optimal speed, reliability, scalability, and user satisfaction.

Category	Objective	Measurability	Target
Performance (Speed & Responsiveness) A responsive application is crucial for user engagement. The aim is to minimize latency and provide a seamless experience.	Achieve sub-2-second load times for both web and mobile platforms. User interactions (e.g., task management, calendar updates) should exhibit a response time of under 500ms. AI assistant responses should be delivered within 3 seconds.	Load times will be assessed using Lighthouse. Application programming interface (API) response times will be tracked via Postman.	95% of user interactions should complete within 500ms. AI responses should demonstrate 85% accuracy and be generated within the 3-second target.
Reliability & Uptime Consistent availability and data integrity are paramount.	Maintain a system uptime of 99.5% or greater. The application should automatically recover from unexpected crashes within 10 seconds. Data loss prevention will be addressed through the saving of data through every process.	Uptime will be monitored using AWS CloudWatch. System errors and downtime will be logged and analysed via Sentry.	Zero data loss incidents per month.\ No security vulnerabilities. No breaches of data security.
Scalability & Load Handling The application must accommodate growth in user base and data volume	Support up to 100,000 concurrent users without performance degradation. The AI assistant should handle a minimum of 1,000 concurrent queries without increased delays.	Load testing will be conducted using tools such as Apache JMeter or Gatling. Database performance will be analysed using SQL Profiler.	Response time degradation under peak load should not exceed 10%. CPU utilization should remain below 80% under high demand.
User Satisfaction & Experience	Achieve a user satisfaction rating of 90% or higher regarding UI navigation. Synchronization of to-do lists, calendars, and planners should occur within 1 second. The AI assistant should provide accurate responses with a target accuracy of 85%.	User feedback will be collected through surveys and feedback forms. AI accuracy will be assessed through user satisfaction ratings of AI-generated responses.	Positive feedback from user surveys. AI responses should demonstrate 85% accuracy and be generated within the 3-second target.

1.5 — IPO (Input-Process-Output)

Develop an IPO model that outlines the application's inputs, processes, and outputs.

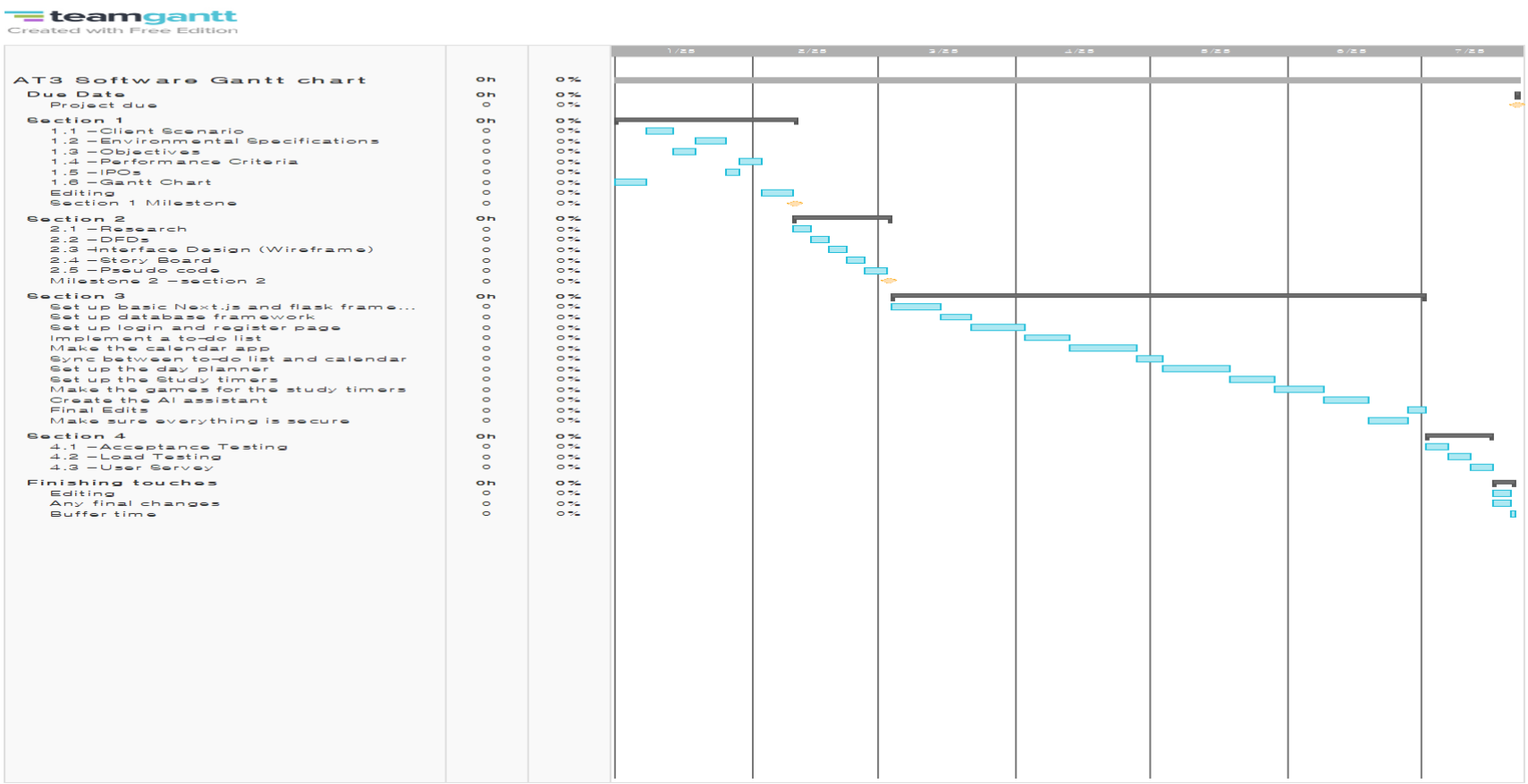
Input	Process	Output
Login (Email and Password)	Check email is valid, match email and password to validate through SHA256. Create Session tokens.	Errors: Display an error code Email or password is invalid: Display error If Correct: Go to home page
Register (Email, Name, Password)	Create a new user. Check the passwords match and the email/ name/ password is valid.	Errors to be displayed if user or email are not unique and if the password doesn't match. If no errors, redirect to login page for security
Create Task (Task, priority, complete, description, Due Date)	Create a new task for the user with valid inputs. Assign it to a database for that user. If due date = True, process that date and task into calendar	Redirect to page showing the task. If due date has been included, show in calendar. If no task or priority display error.
Update Task (Complete)	Update the database with a Boolean response	Update to show task has been completed
Delete Task	Delete the task from the database.	Remove the task from the user's view.
Create Event (Name, Date (time), location, repeat)	Check that there is a Name and date, other info is optional. Add to database under username.	Display on the calendar with event details. If no Name or date, display error.
Edit Event (Name, Date (time), location, repeat)	Check valid name and date, adjust in database.	Display updated task in the calendar
AI (Input/ prompt)	Send the prompt to the large language modal (LLM) to seek a response to the user. Push to a conversation history in a database.	Deliver a message to be shown back to the user. If out of credits display error If LLM error, display monetised version of error.
Create Daily plan Blocks (Task, Duration, Starting time)	Check all valid inputs and add them into the daily plan. Add them into a temporary database that will be deleted at the end of every day. Assign a break underneath the plan (1/3 of duration)	Post a block in the daily schedule for the task and the duration at that time. Add in a block for a break underneath the study block.
Edit Daily plan block (Task, Duration, starting time) Delete Block	Bring pop up that displays the task and then allow for editing. Validate and confirm edits. Allow the ability to delete the block.	Post the updated block in the daily schedule for a task. If deleted, remove the block from the view of the user.

Input	Process	Output
Start Timer (Free study, start Day)	Free Study: User chooses a time to study. Create a break, 1/3 of the time they set Start Day: Goes through the daily plan with automated breaks already adjusted.	Bring up the timer for the allocated time, with a clock running counting down. Once timer ends, bring up break screen.
Breaks (Games, own time)	Games: Process the choices of game/s Own Time: Start timer	Bring up the game that is chosen and start the timer for 1/3 of the duration of the study period or start the timer for own time.
Settings and Profile Management	Open the settings menu	Display the menu for the user
Dark Mode	Switch the theme of the website to dark mode or light mode depending on the preference.	Update the page for the viewers preference.

1.6 — Gantt Chart

Create a Gantt chart to outline the project timeline. The chart should include all major milestones, tasks, and deadlines. It should also visually represent the project schedule, showing each task's start and end dates and the dependencies between tasks.

NOTE: This Gantt chart was created at the start and has been modified to reflect the percentage and completion rates for each task.



Milestone 1: This should be completed by term 1, week 2.

Section 2 - Research and Planning (10 Marks)

2.1 — Research

Conduct thorough research to gather information relevant to the project. This should include market analysis, user research, competitor analysis, and other pertinent data. Summarise the findings and explain how they will inform the project's development.

Market Study

Industry Overview

The market for productivity and organisation applications has undergone rapid growth within the last few years driven by the need for digital tools to help individuals organise their time effectively. The impact of remote learning or workspaces has driven a need for people to use tools so they can blend tasks and schedules more efficiently. Applications such as Notion, Todoist, Google Calendar, and Microsoft Planner already exist and have functions that are useful to both students and working professionals. However, most applications that are available are only addressing certain areas of organisation, leaving a gap for a more comprehensive offering that incorporates a to-do list and event calendars, study timers, and AI assistance.

Key Trends and Opportunities

Key trends that are shaping students and the work force include:

- accessibility to information, particularly though the use of AI and LLM. This enables people to get instant responses to questions and provides guidance or input to decisions.
- a focus on reward and recognition as a motivator for productivity improvement. This includes creating an environment where people can reward themselves for achieving milestones
- an expectation of cross device synchronisation in an ever increasingly connected environment
- a drive for more user friendly and intuitive applications that are focused on user experience
- the need for tools to help people manage their increasing complexity in their lives. This is combined with the increased levels of autonomy where people are studying or working more in their "own" environment rather than group or office environments. As such there is a need for the ability to self-manage and self-regulate one's activities

User Research

Target Audience

The audience comprises students, professionals, and individuals needing help with organisation and planning. Students at high school or university could benefit from the ability to schedule study time, deadlines, and tracking assignments. Professionals or adults could benefit from having a tool that enables efficient task calendar to plan tasks, meetings, and even personal responsibilities to maintain a good work-life balance. Students and adults alike appreciate systems that help them plan their daily activities. This is more so in the case where they may have attention or focus difficulties such as those who have executive dysfunction, for example, ADHD.

Main User Needs

The main user needs are:

- A seamless task management system that allows users to create, edit, and view their tasks consists of a simple interface with a scheduling feature that can be integrated with a calendar system for better time management.
- Customisable study timers are required to enable users to set up their own focus study time and scheduled breaks to enhance efficiency.
- AI arrangement and study help could offer immediate assistance for queries about organisation, study issues, and productivity ideas.
- Cross-device access so that users can access their schedules and tasks through multiple devices without disruption would be a relevant feature.
- Simple easy useable and intuitive interface, which should make the application usable for a variety of users.

Competitor Analysis

Key Competitors

The competition in the productivity sector is filled with applications like Google Calendar, Todoist, Notion, Trello, and gamification-based Forest and Pomodoro, with many firmly entrenched as market leaders. All have strong scheduling ability and are widely used because of their integration with other services. For instance, Google services such as Google Calendar and Tasks can link together but are still separate applications.

Strengths and Weaknesses:

Competitor	Strengths	Weaknesses
Todoist	Easy user interface, multidisciplinary approach, emphasises task importance.	Lacks advanced task management, no calendar services.
Notion	Highly customisable, collaborative workspaces, flexible.	Complex and overwhelming for new users, hard to learn.
Trello	Masterful visual task board, excellent for project management.	Not great for personal daily plans or organisation.
Gamified Timers (e.g., Forest, Pomodoro)	Shows focus and productivity, encourages productivity.	Limited functionalities, not helpful without a deeper toolbox for organisation.
Google Calendar	Great for scheduling and coordinating events with others.	Does not have solid task management.

Price ranges:

App	Free Plan	Paid Plans	Key Features for Paid Plans
Todoist	Yes	From $4 - $6 AUD	Increased projects, collaborators, activity history, calendar layout, task duration, AI assistant (Pro)
Notion	Yes	From $10 - $20 AUD (or custom for Enterprise)	Unlimited blocks for teams, advanced permissions, larger file uploads, more powerful databases
Trello	Yes	From $5 - $17.50 AUD	Unlimited boards, advanced checklists, custom fields, additional views (Timeline, Calendar, Dashboard)
Forest	Limited*	Often a one-time purchase of around $4 - $7 AUD	Cross-device access, syncing
Pomodoro Apps	Yes (many)	Varies, some have in-app purchases for premium features (e.g., $1.99 - $9.99 AUD for lifetime)	Ad-free experience, customisation, advanced reporting
Google Calendar	Yes	Included with Google Workspace plans (from ~$9.90 AUD/month)	More storage, larger video meetings, advanced security, business features (via Google Workspace)

Summary of Findings

Main Insights

The need for AI input in productivity applications is high as users derive benefits from intelligent assistance, giving real-time recommendations and support. Most of the existing tools are not integrated with a single productivity application. The bulk of them are either specialists in task management, calendar, or time management, with few having an integrated experience. Customisable and simplicity are keys to satisfaction, as the user prefers applications with a degree of flexibility balanced by a simple interface. Gamification works well for engagement; study timers with built-in rewards help users stay focused and productive. Moreover, users expect to synchronise across devices with ease, allowing switching without losing any progress.

In short, the vision for the application is to fill a gap in the market that provides an integrated solution with inbuilt reward mechanisms ultimately driving productivity and output.

Impact on Development of the Project

The focus of the application would be on:

- seamless user interface and usability for ease of use.
- integration of to do list and a calendar, which interact together, to help users manage their schedules.
- centred on a reward and recognition framework for performance output
- cross device support and operability to see plans whenever, wherever
- incorporation of AI to aid in the development of a productive study environment

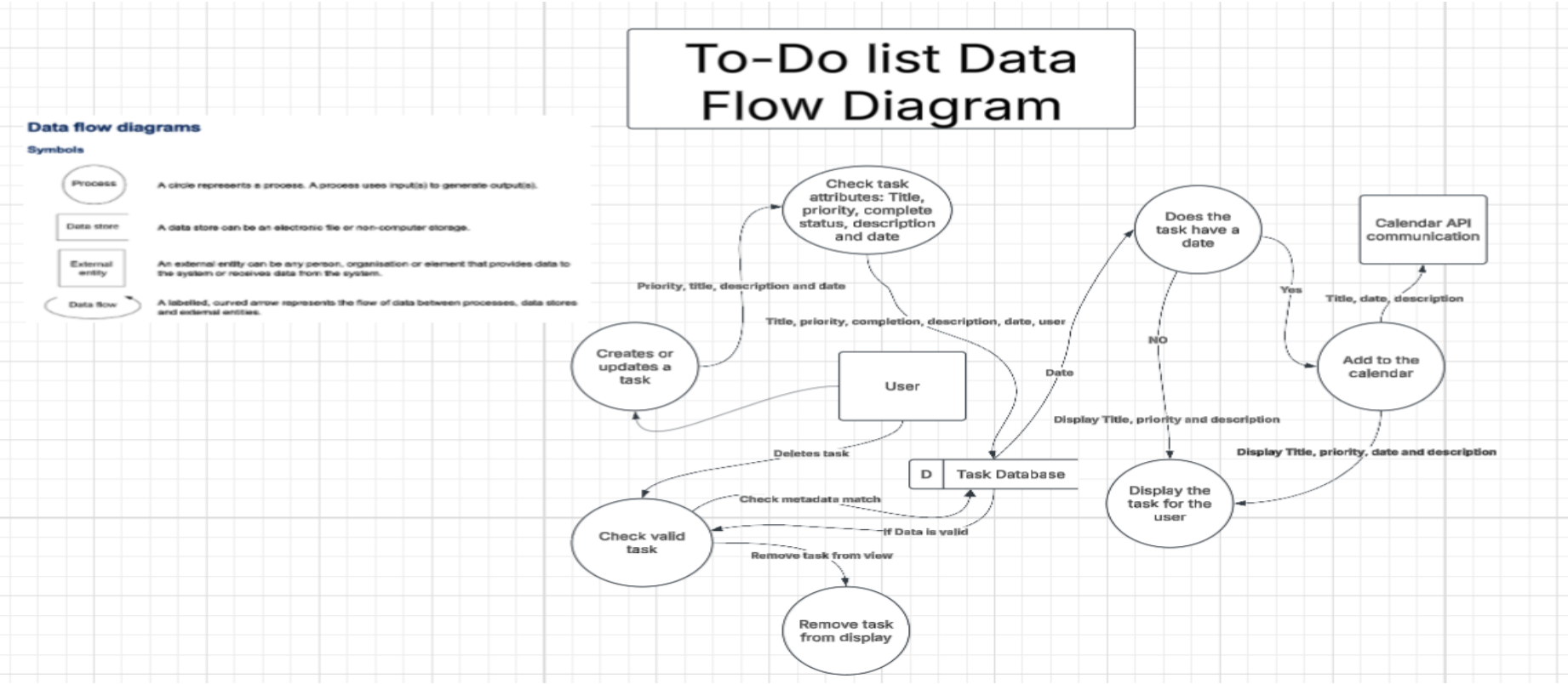
Overall improving organisation and enhancing productivity, allowing for students and professionals to perform better.

2.2 — Data Flow Diagram (DFD)

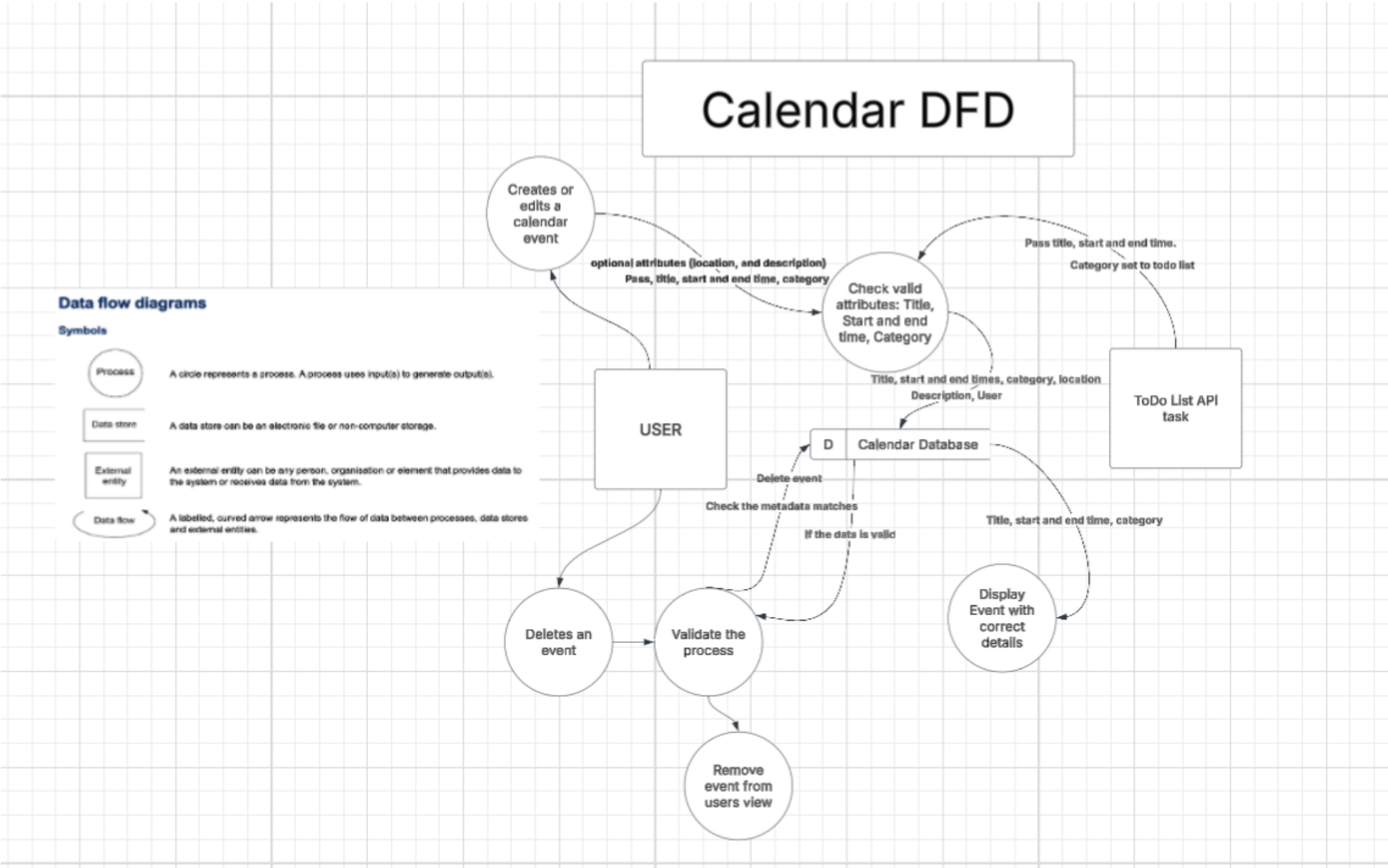
Design a DFD for each feature of your project to illustrate how data will flow through the system. The DFD should show the interactions between inputs, processes, and outputs. Provide detailed diagrams and explanations for each feature.

DFD symbols key came from: https://curriculum.nsw.edu.au/learning-areas/tas/software-engineering-11-12-2022/overview. As per the rubric, note that 3 DFD's have been outlined below which are focussed on those areas which are more complex in nature. It is noted that there would be other DFD's which would cover areas such as onboarding of new users.

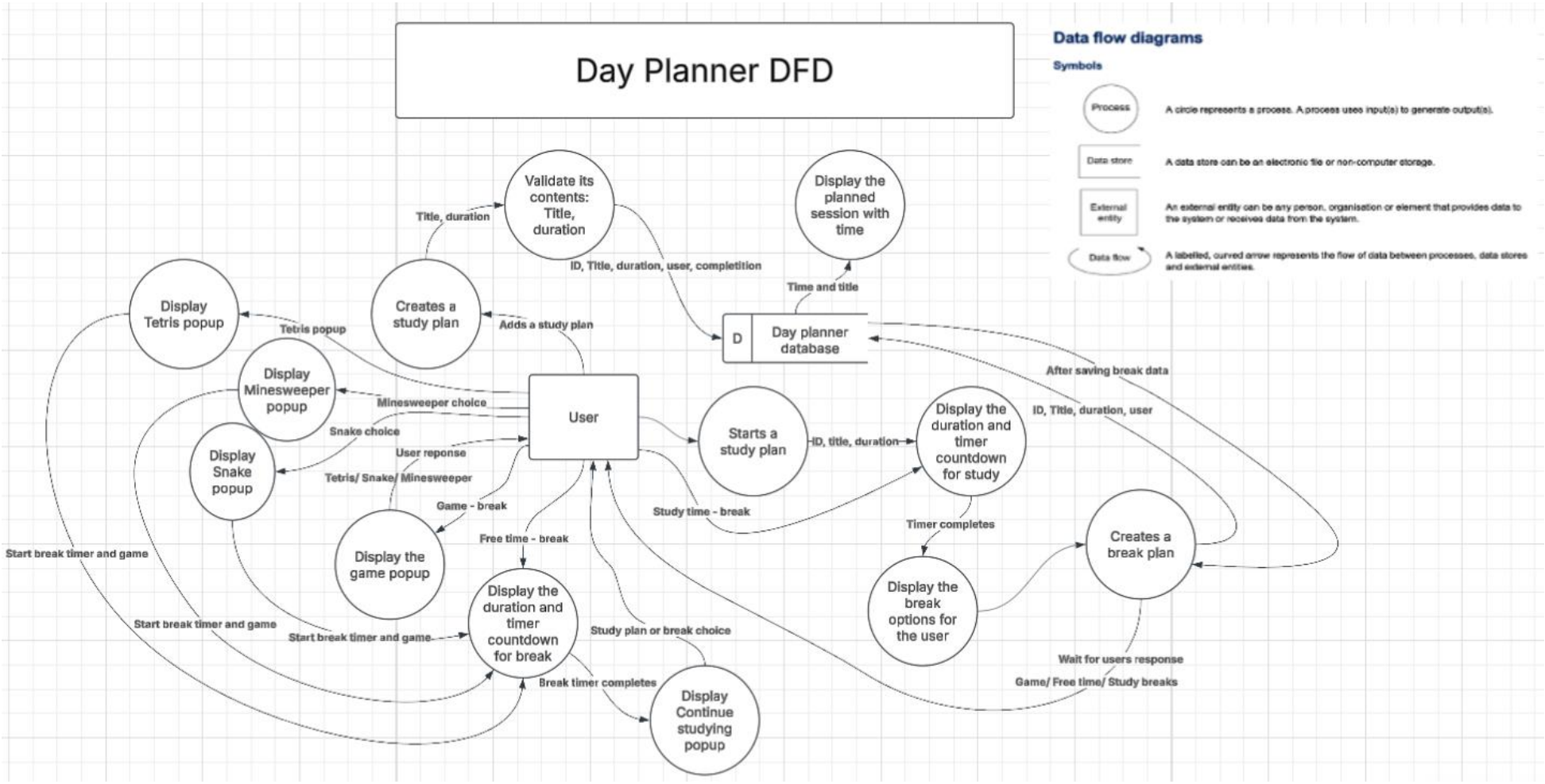
DFD 1 – To-do List



DFD 2 - Calendar



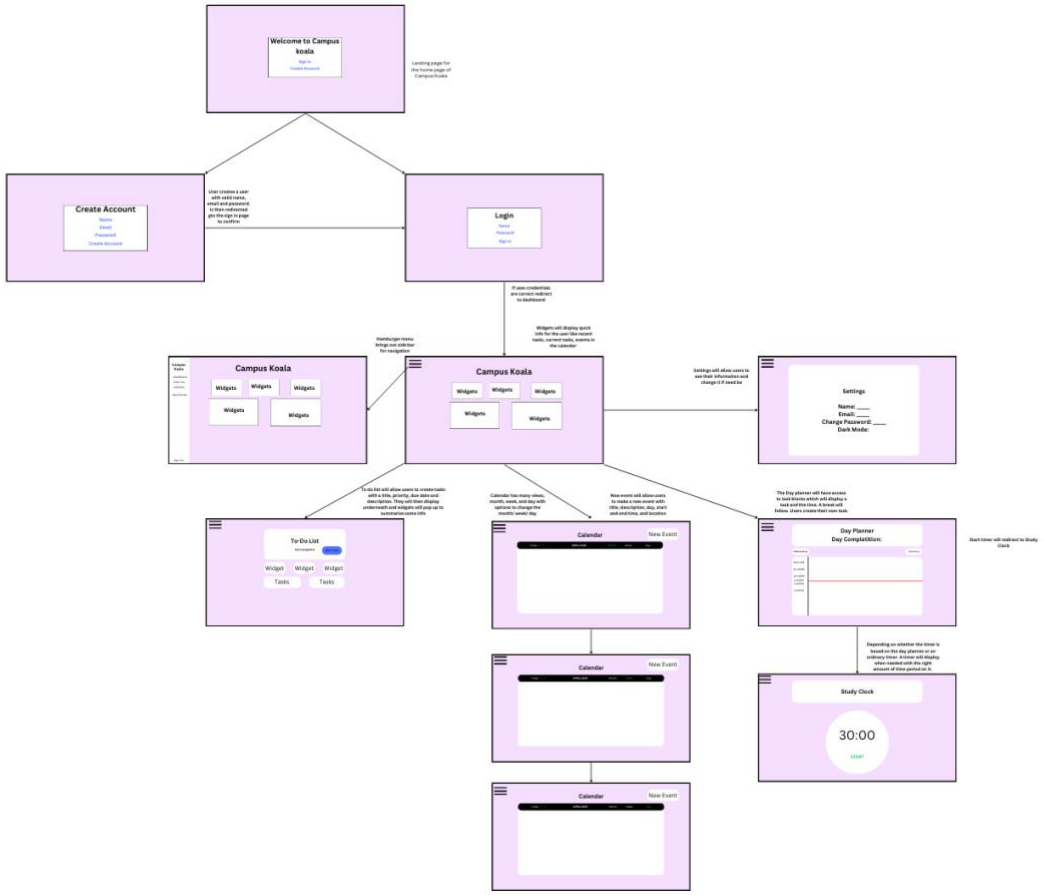
DFD 3 – Day Planner + Timers



2.3 — Interface Design (Wireframe)

Develop wireframes for the application's user interface. These wireframes should visually represent the application's layout and design. Include annotations and explanations for each element of the interface.

https://www.canva.com/design/DAGgkYzOXqA/01cMGcXhXOAygX-oMlPTnQ/view?utm_content=DAGgkYzOXqA&utm_campaign=designshare&utm_medium=link2&utm_source=uniquelinks&utlId=he4bf0b7e8c – Version online for easier use if the below image does not load.

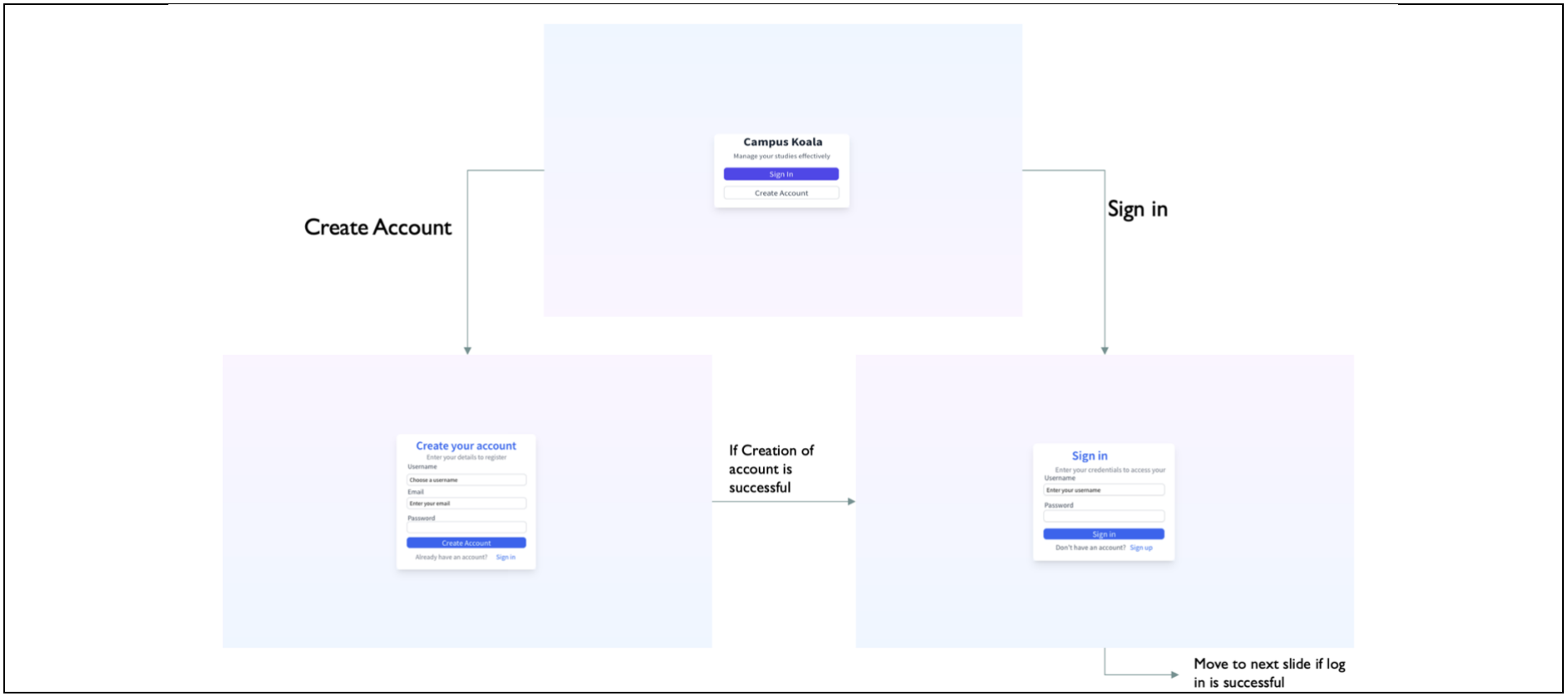


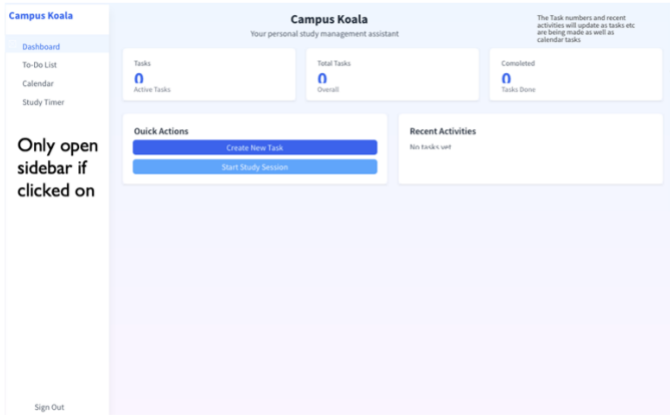
2.4 — Storyboard

Create a storyboard to outline the user journey through the application. The storyboard should include key screens, user actions, and interactions. Provide a narrative to explain how users will navigate the application and achieve their goals.

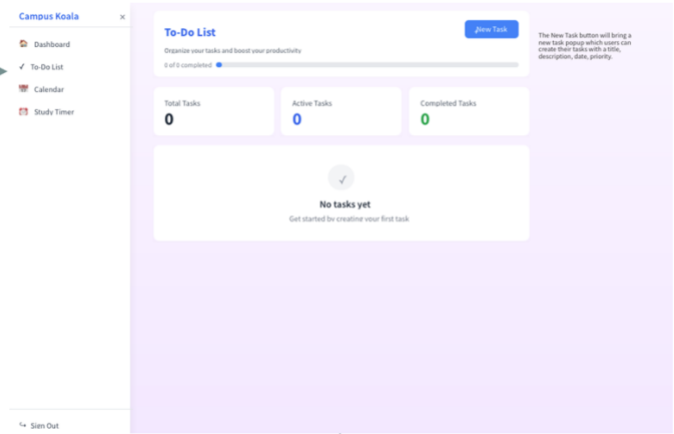
Please use the link below to access the storyboard should the images below not be easily digested.

https://cgsacteduau-my.sharepoint.com/:p:/g/personal/49132_cgs_act_edu_au/EQmwbbHcG7xAlHC_UcDmIDYBQEVK_n-Ozf6Gj-tdwhrI-g?e=u6CKEQ&nav=eyJzSWQiOjI1NiwiY0lkIjoxNjU0MDY2NzA1fQ

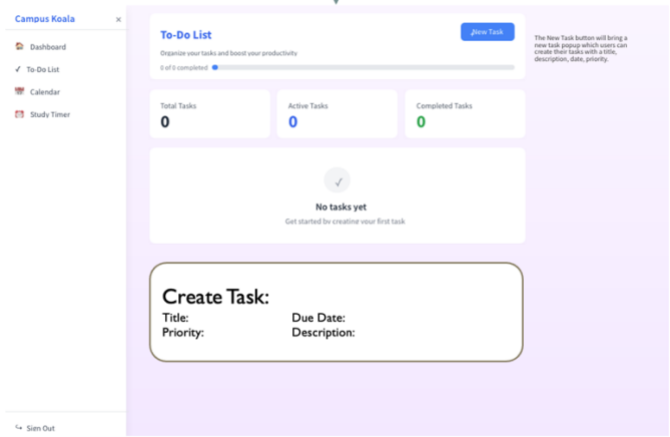




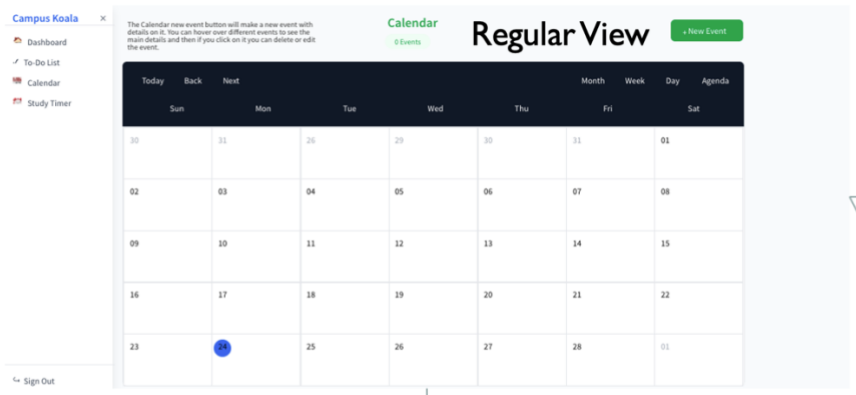
When To-do list of create new task is clicked



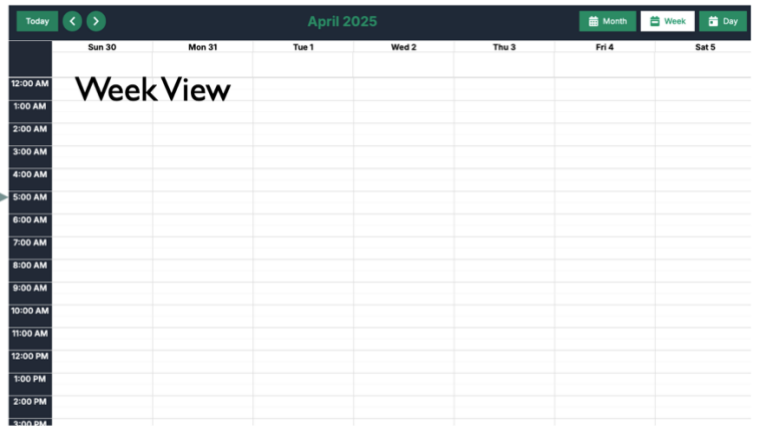
New task is clicked



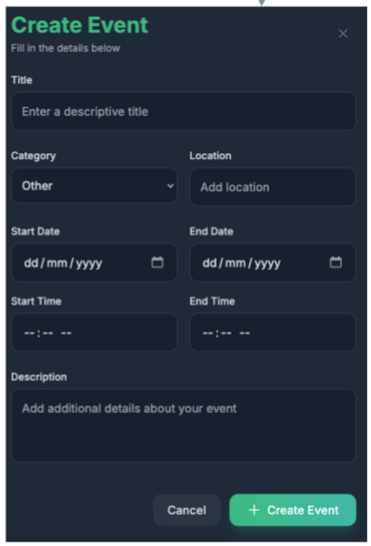
Dashboard + To-do List



Week View is clicked

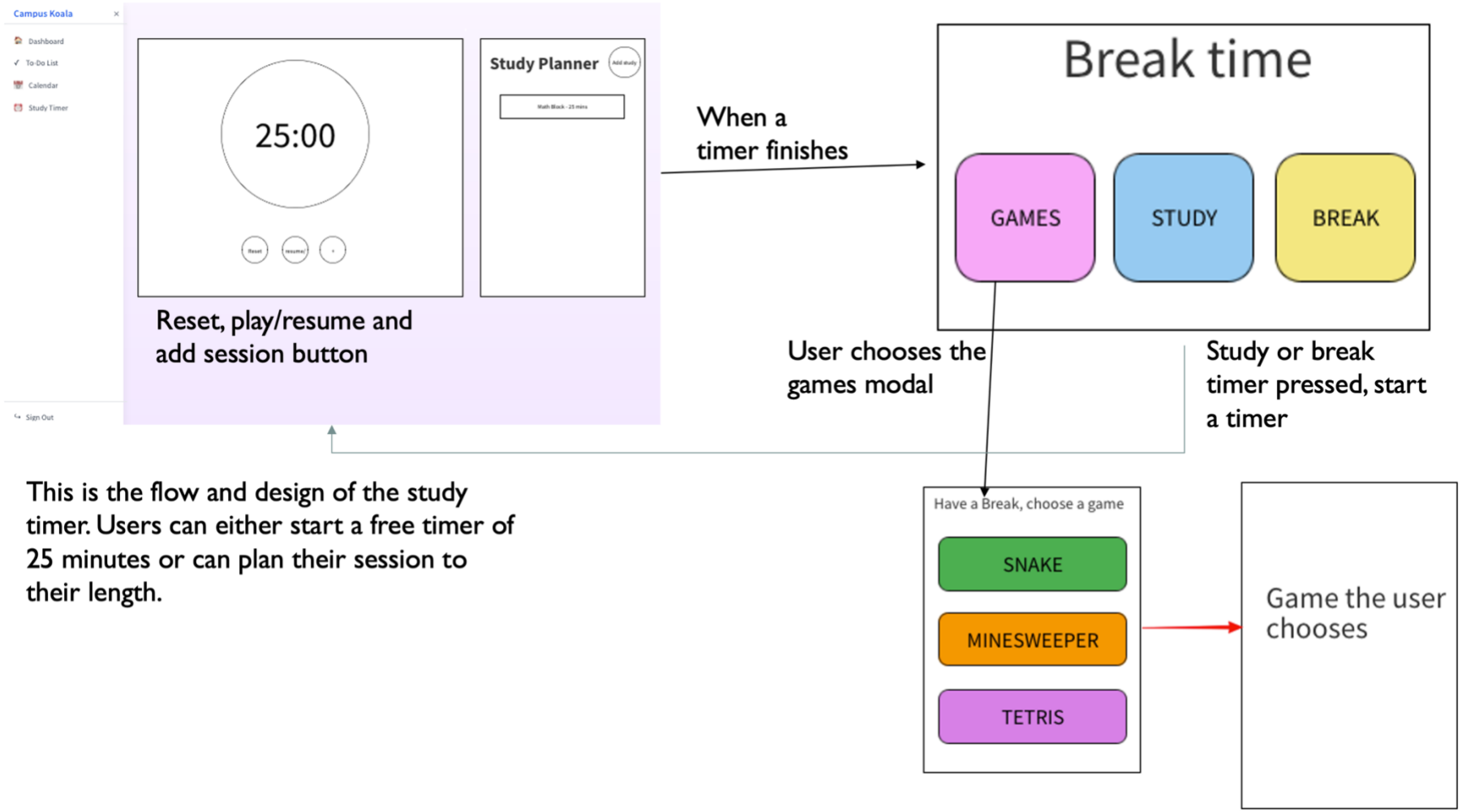


New Event Clicked



Create Event pop up

Day View is clicked



This is the flow and design of the study timer. Users can either start a free timer of 25 minutes or can plan their session to their length.

2.5 — Pseudocode

Write five pseudocode algorithms for critical functions and algorithms in the web application's backend. These pseudocode algorithms should provide a clear foundation for coding the application. Include detailed explanations and comments for each algorithm.

Algorithm 1 – Login/ Authentication

```
# User Login Process
# This function handles user authentication by validating credentials and establishing a session
# Parameters:
# - username: The user's unique identifier (email or username)
# - password: The user's password (will be hashed for comparison)
BEGIN Login(username, password)
    # Initialize error state
    SET error to none

    # Validate input fields
    IF username or password is empty THEN
        SET error to "All fields must be filled"
        DISPLAY error
    END IF

    TRY
        # Send authentication request to the server
        SEND POST request to '/api/login' with data username, and password

        # Retrieve user data from database
        SET user = username

        # Verify user exists
        IF user not found THEN
            # Generic message to avoid username enumeration
            DISPLAY error "Invalid username or password"
        END IF

        # Hash the provided password for comparison
        SET hashedPassword to the SHA256 of password

        # Verify password matches stored hash
        IF user password_hash does not match hashedPassword THEN
            # Generic message to avoid timing attacks
            DISPLAY error "Invalid username or password"
        END IF

        # Creates a JWT token with user information
        SET access_token = JWT encode user.id and user.username

        # Stores the token in local storage for subsequent requests
        SET localStorage.token = access_token

        # Redirects to dashboard
        Redirect to dashboard

    CATCH error THEN
        # Catches any errors      You, 6 minutes ago • Uncommitted changes
        DISPLAY error "Login failed. Please check your credentials."
    END TRY
END Login
```

Algorithm 2 - Register

```
# User Registration Process
# This function handles new user registration with comprehensive validation and security measures
# Parameters:
# - username: Desired username (must be unique)
# - email: User's email address (must be valid and unique)
# - password: User's chosen password (must meet complexity requirements)
BEGIN Register(username, email, password)
    # Initialize error state      You, 5 minutes ago • Uncommitted changes
    SET error to none

    # Validate all required fields are provided
    IF username or email or password is empty THEN
        SET error to "All fields must be filled"
        DISPLAY error
    END IF

    # Password validation rules
    # Minimum length requirement
    IF password length < 8 THEN
        SET error to "Password must be at least 8 characters long"
        DISPLAY error
    END IF

    # Require at least one uppercase letter
    IF password has no uppercase letter THEN
        SET error to "Password must include at least one uppercase letter (A-Z)"
        DISPLAY error
    END IF

    # Require at least one lowercase letter
    IF password has no lowercase letter THEN
        SET error to "Password must include at least one lowercase letter (a-z)"
        DISPLAY error
    END IF

    # Require at least one number
    IF password has no number THEN
        SET error to "Password must include at least one number (0-9)"
        DISPLAY error
    END IF

    # Require at least one special character
    IF password has no special character THEN
        SET error to "Password must include at least one special character (@$!%*?&)"
        DISPLAY error
    END IF

    TRY
        # Sends a request to the API that hanndles the task with relevent details
        SEND POST request to '/api/register' with data username, email, and password

        # Checks if the username already exists`
        IF User with username exists THEN
            DISPLAY error "Username already exists. Please choose a different username."
        END IF

        # Checks if the email already exists
        IF User with email exists THEN
            DISPLAY error "Email address is already registered. Please use a different email or try logging in."
        END IF

        # Hashes the password
        SET hashedPassword to the SHA256 of password

        # Creates a new user object
        SET newUser to Username, email, and hashedPassword

        # Saves the new user to the database
        SAVE newUser to database

        DISPLAY success

    CATCH error THEN
        # Catches any errors
        DISPLAY error "Registration failed. Please try again."
    END TRY
END Register
```

Algorithm 1 – To-do List (create task)

```
# Todo List Module
# This module handles the creation and management of todo list tasks
# Includes validation, error handling, and database operations
BEGIN TodoList

    # Create a new task in the todo list
    # Parameters:
    # - title: Short description of the task (required)
    # - date: Due date for the task (optional)
    # - description: Detailed information about the task (optional)
    # - priority: Importance level of the task (required: 'low', 'medium', 'high')
    FUNCTION CreateTask(title, date, description, priority)
        TRY
            # Validate required fields
            IF title or priority is empty THEN
                DISPLAY error "Title and priority are required"
            END IF

            # Sends a request to the API that hanndles the task with relevent details
            SEND POST request to '/api/tasks' with data title, date, description, priority

            # Creates a new task with the relevent details
            CREATE new Task with title, date, description, priority, completion, user_id

            # Saves the task to the database
            SAVE task to database

            DISPLAY New task
            ⌘I for Command, ⌘L for Cascade
        CATCH error THEN
            # Catches any errors
            DISPLAY error "Failed to create task. Please try again."
        END TRY
    END FUNCTION
END TodoList
```

Algorithm 2 – Calendar (Create Event)

```
# Calendar Event Creation
# This function handles the creation of new calendar events with validation and error handling
# Parameters:
# - title: The title of the event (required)
# - start_time: When the event begins (required, in ISO 8601 format)
# - end_time: When the event ends (required, in ISO 8601 format)
# - description: Optional detailed description of the event
# - category: Optional category for the event (e.g., 'Work', 'Personal')
# - location: Optional physical or virtual location of the event
BEGIN FUNCTION CreateEvent(title, start_time, end_time, description, category, location)      You, 7 days ago

    TRY
        # Validate required fields
        IF title, start_time, or end_time is empty THEN
            DISPLAY error "Title, start time, and end time are required"
        END IF

        # Parse datetime strings into datetime objects
        SET parsed_start_time TO Parse datetime from start_time
        SET parsed_end_time TO Parse datetime from end_time

        # Validate datetime format
        IF parsed_start_time is invalid OR parsed_end_time is invalid THEN
            DISPLAY error "Invalid datetime format"
        END IF

        # Validate time range
        IF parsed_end_time LESS THAN OR EQUAL TO parsed_start_time THEN
            DISPLAY error "End time must be after start time"
        END IF

        # Authenticate user
        SET user_id TO Extract user ID from JWT token

        # Verify authentication
        IF user_id is invalid THEN
            DISPLAY error "Authentication failed"
        END IF

        # Creates a new event object
        CREATE new CalendarEvent with title, start_time, end_time, description, category, location, user_id

        # Saves the event to the database
        SAVE event to database

        DISPLAY "Event created successfully"

    CATCH error THEN
        # Catches any errors
        DISPLAY error "Failed to create event. Please try again."
    END TRY
END FUNCTION
```

Algorithm 3 – AI API CONNECTION

```
# AI ChatBot Response Generation        You, 9 minutes ago • Uncommitted changes
# This function handles the core chat functionality by processing user input and generating AI responses
# Parameters:
# - prompt: The user's input message
# - conversation_id: Unique identifier for the conversation thread (optional)
# - context: Additional context for the conversation (optional)
BEGIN FUNCTION GenerateResponse(prompt, conversation_id, context)

    # Get and validate user input
    SET prompt to users input

    # Check if prompt is provided
    IF prompt is NONE THEN
        DISPLAY "Error: Prompt is required"
    END IF

    # Handle conversation management
    IF conversation_id is NONE THEN
        # Start a new conversation if no ID is provided
        SET conversation_id TO newConversation
    ELSE
        # Retrieve existing conversation
        SET conversation TO conversation(conversation_id)
        IF conversation is "NOT_FOUND" THEN
            DISPLAY "Error: Conversation not found"
        END IF
    END IF

    # Prepare context for the AI by combining conversation history and new prompt
    SET context to conversation_id + prompt

    # Generate AI response using the prepared context
    SET response TO GenerateAIResponse(context)

    # Store both user message and AI response in conversation history
    STOREMessage(conversation_id, "user", prompt)
    STOREMessage(conversation_id, "assistant", response)

    # Format the complete response with conversation context
    SET formatted_response TO response, conversation_id, history

    # Return success message
    DISPLAY "Response generated successfully"

    # Error handling
    ON ERROR
        DISPLAY "Error: Failed to generate response. Please try again."
    END ON ERROR
END FUNCTION
```

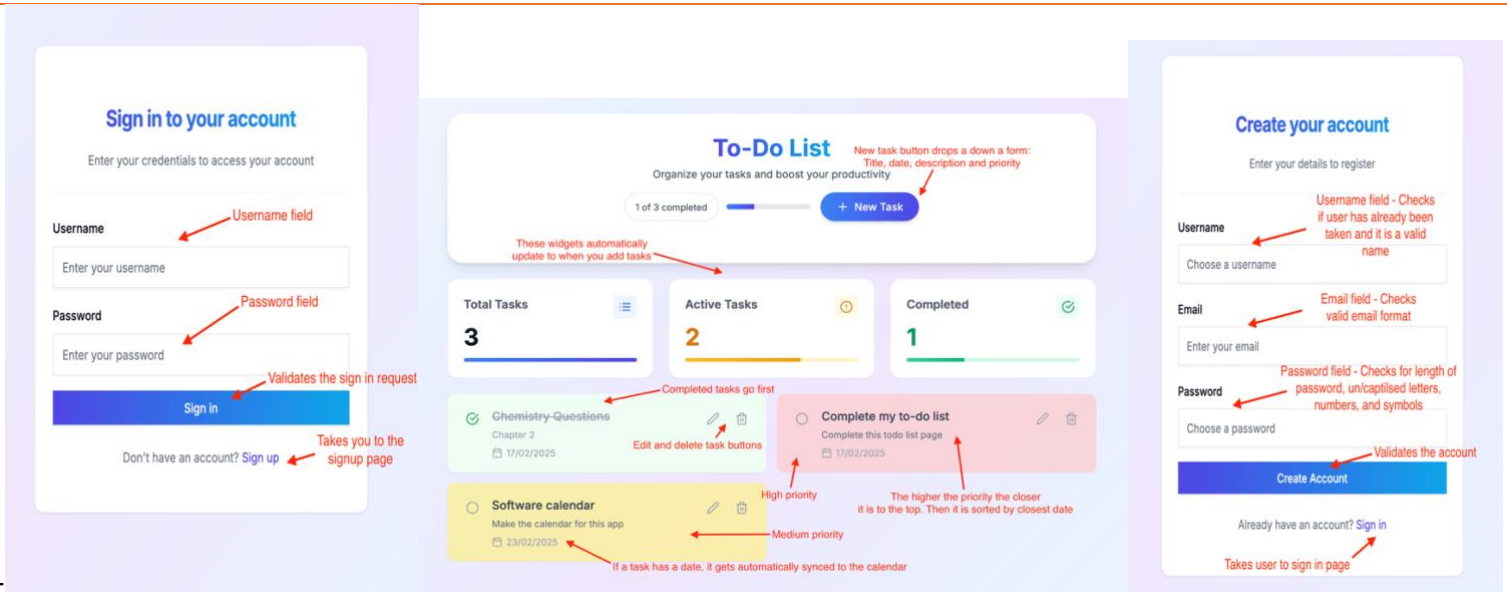
Milestone 2: This should be completed by term 1, week 5.

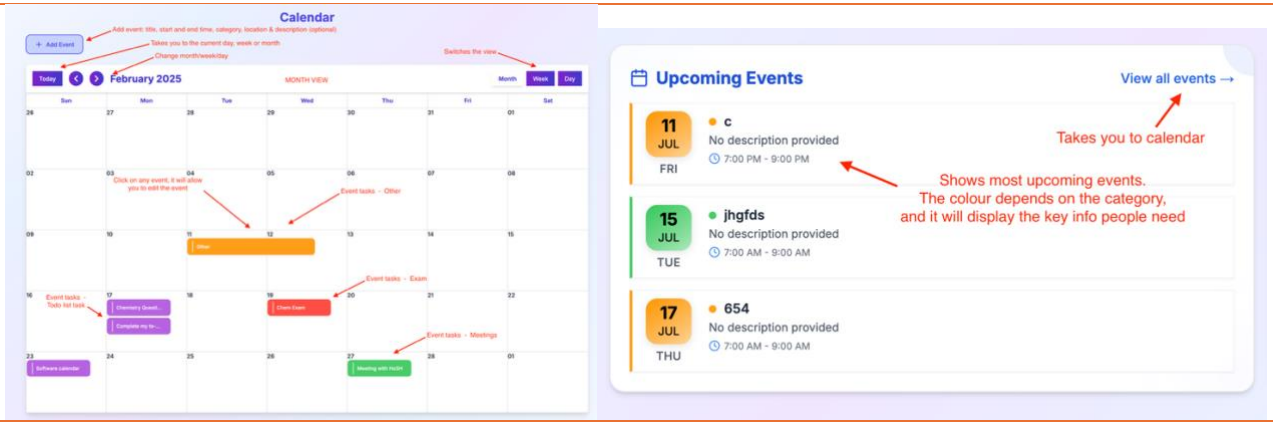
Section 3.1 - Development (10 Marks)

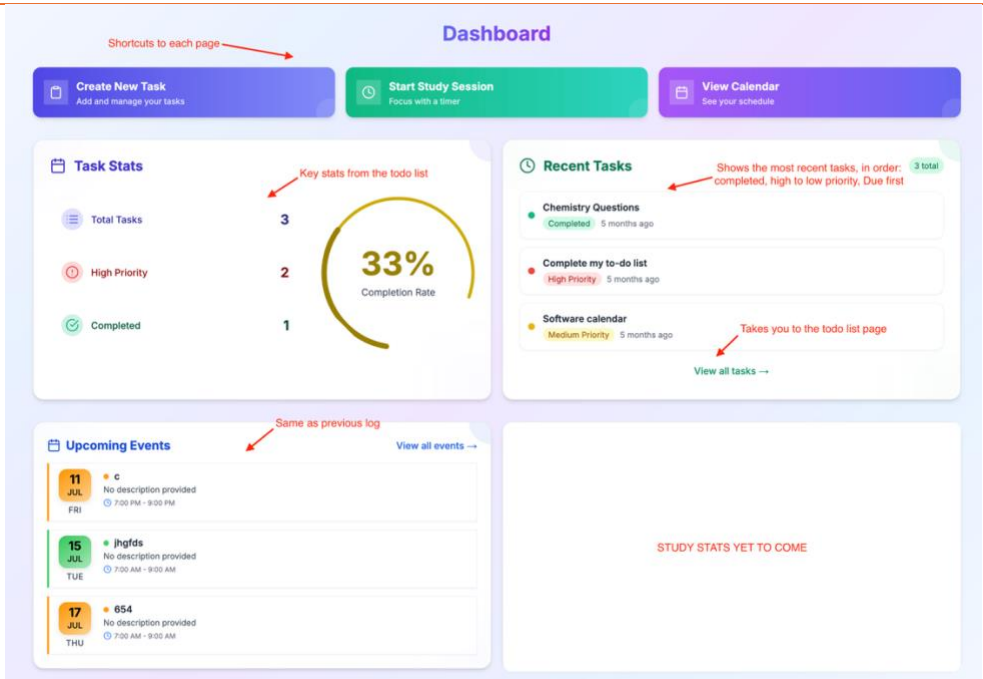
3.1.2 — Logbook

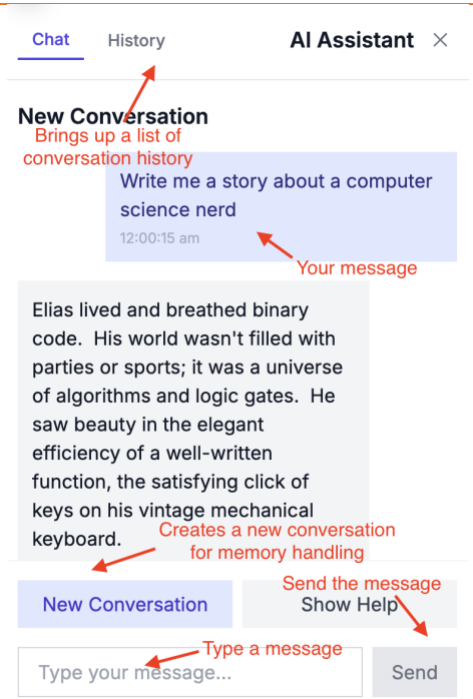
Develop a logbook for tracking changes and implementing significant features and changes/additions to your web application. The logbook should follow the styling guide provided in the course specs. Include entries for each significant development activity, with dates, descriptions, and outcomes.

Date:	8/2/25
Title:	Initial commit and creation of Campus Koala
Author:	Jake Handel
Tasks Description:	Made a scaffold and a skeleton of the website, plain to very limited functionality.
Image:	N/A
Logbook Entry:	Progress since the last entry: N/A Task Achieved: Scaffolding a website for development Stumbling blocks or issues encountered and how they were managed: Connection with Database Possible approaches for upcoming tasks: Todo list functionalities and database connection Reflective comments: good start, develop more CSS and UI contributions to it.
Resources:	N/A

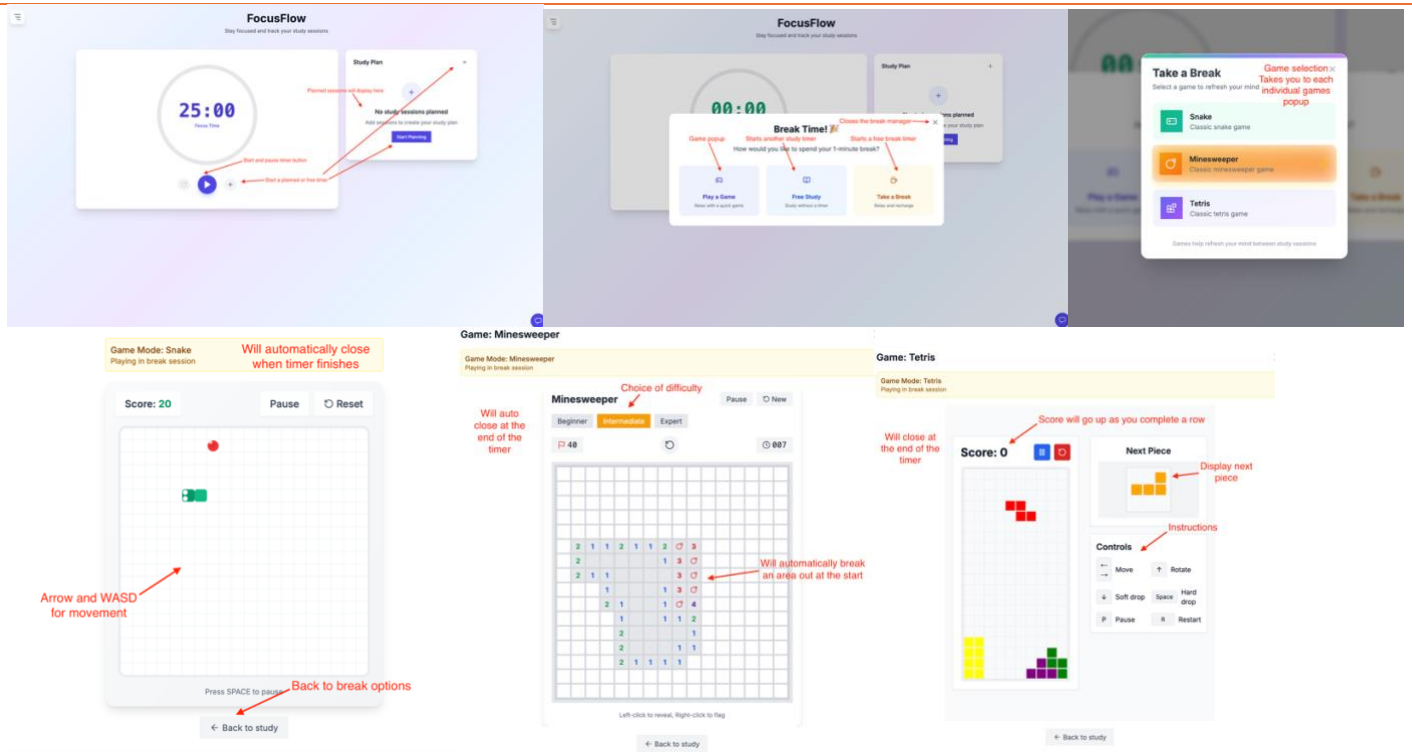
Date:	17/2/25
Title:	Finalising the To-do list functionalities and the authentication of the user
Author:	Jake Handel
Tasks Description:	Make, edit, delete, complete to-do list with backend connection. Authentication of users and key usage.
Image:	 -
Logbook Entry:	Progress since the last entry: Created a to-do list and made the login page work Task Achieved: Todo list and authenticate users Stumbling blocks or issues encountered and how they were managed: Authentication not working, fixed using console logs and understanding how it was flowing. Names and passwords weren't being stored correctly. Possible approaches for upcoming tasks: Calendar, use a library for the calendar and sync with Todo list. Start with basic creation and move up from there. Reflective comments: Great start and flows are going; general UI has improved.
Resources:	N/A

Date:	28/2/25
Title:	Merging the calendar update to main, syncing to dashboard events.
Author:	Jake Handel
Tasks Description:	Create, edit, delete calendar events. Events sync to the dashboard.
Image:	
Logbook Entry:	Progress since the last entry: Made the calendar and edited the dashboard syncing Task Achieved: Calendar, and dashboard syncing Stumbling blocks or issues encountered and how they were managed: Timings would be set for a time and then not be set for that time, making an event session task block. Possible approaches for upcoming tasks: Database syncing if the to-do list has a date, then sync to calendar Reflective comments: Going smoothly, continue updating dashboard for UI
Resources:	React Big Calendar

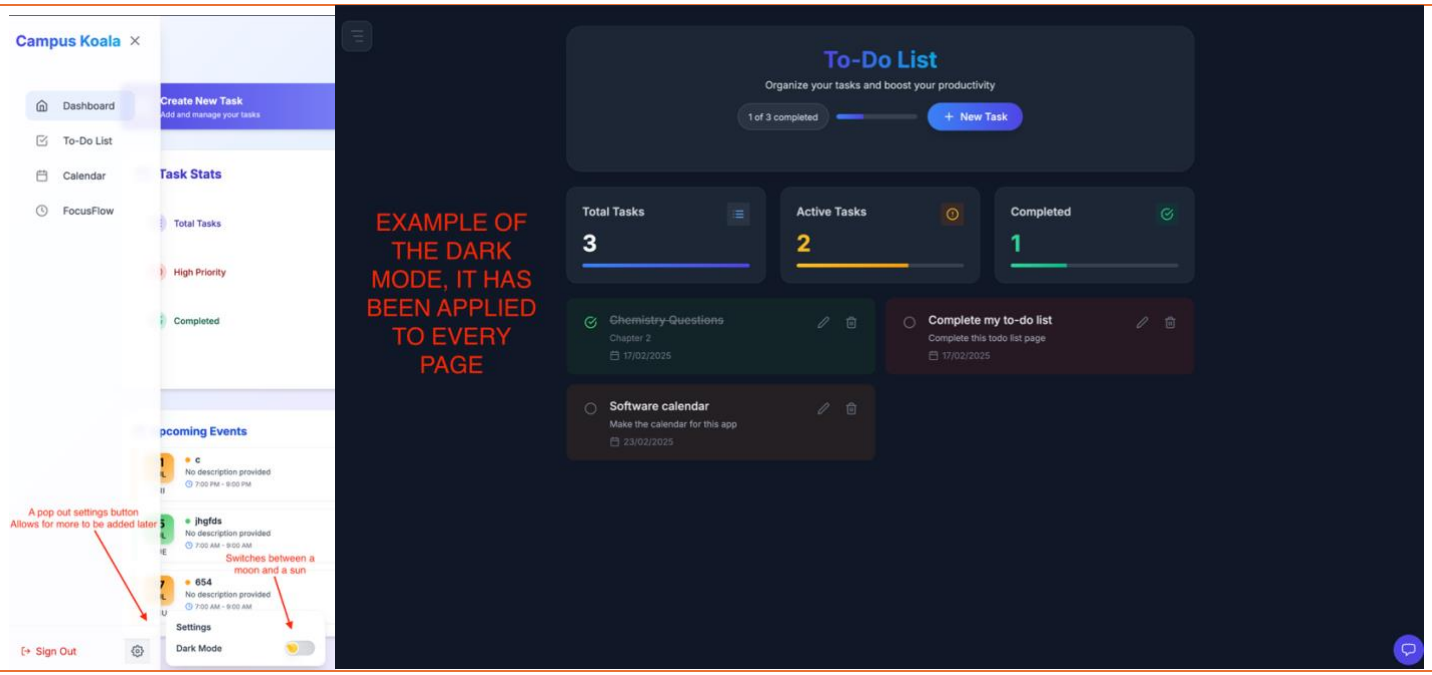
Date:	1/3/25
Title:	Dashboard and to-do list syncing to the calendar
Author:	Jake Handel
Tasks Description:	Making the dashboard more stylistic and to-do list syncs to the calendar
Image:	
Logbook Entry:	Progress since the last entry: Dashboard UI, syncing to-do list to calendar Task Achieved: Dashboard and to-do list syncing Stumbling blocks or issues encountered and how they were managed: Few syntax errors when making the dashboard. Systematic checking. Possible approaches for upcoming tasks: AI, API keys, and version memory, saving chats. Reflective comments: Going well so far, AI next.
Resources:	AI for the circle and dots within the dashboard. (Claud.ai)

Date:	26/4/25
Title:	AI Chatbot
Author:	Jake Handel
Tasks Description:	Making a chatbot with conversation history
Image:	
Logbook Entry:	Progress since the last entry: Made an AI chatbot Task Achieved: AI chatbot created and works Stumbling blocks or issues encountered and how they were managed: Saving conversation history and saving it to backend, solved through numerous tests Possible approaches for upcoming tasks: Solidify the saving of the AI chatbot, move onto study plan. Reflective comments: May need to adjust some of the AI features.
Resources:	Google Gemini API

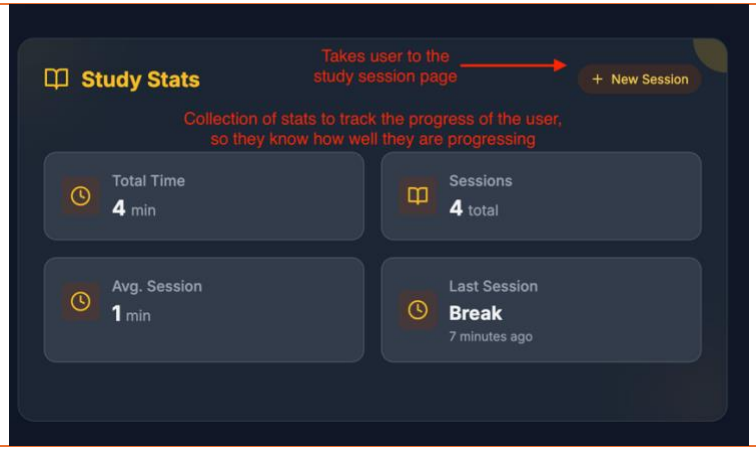
Date:	17/5/25
Title:	Calendar, dashboard updates and starting day planner
Author:	Jake Handel
Tasks Description:	Changing layout and UI of both calendar and dashboard
Image:	N/A
Logbook Entry:	Progress since the last entry: Updating UI and layout Task Achieved: Better user experience Stumbling blocks or issues encountered and how they were managed: None Possible approaches for upcoming tasks: N/A Reflective comments: Good layouts move onto study planner.
Resources:	N/A

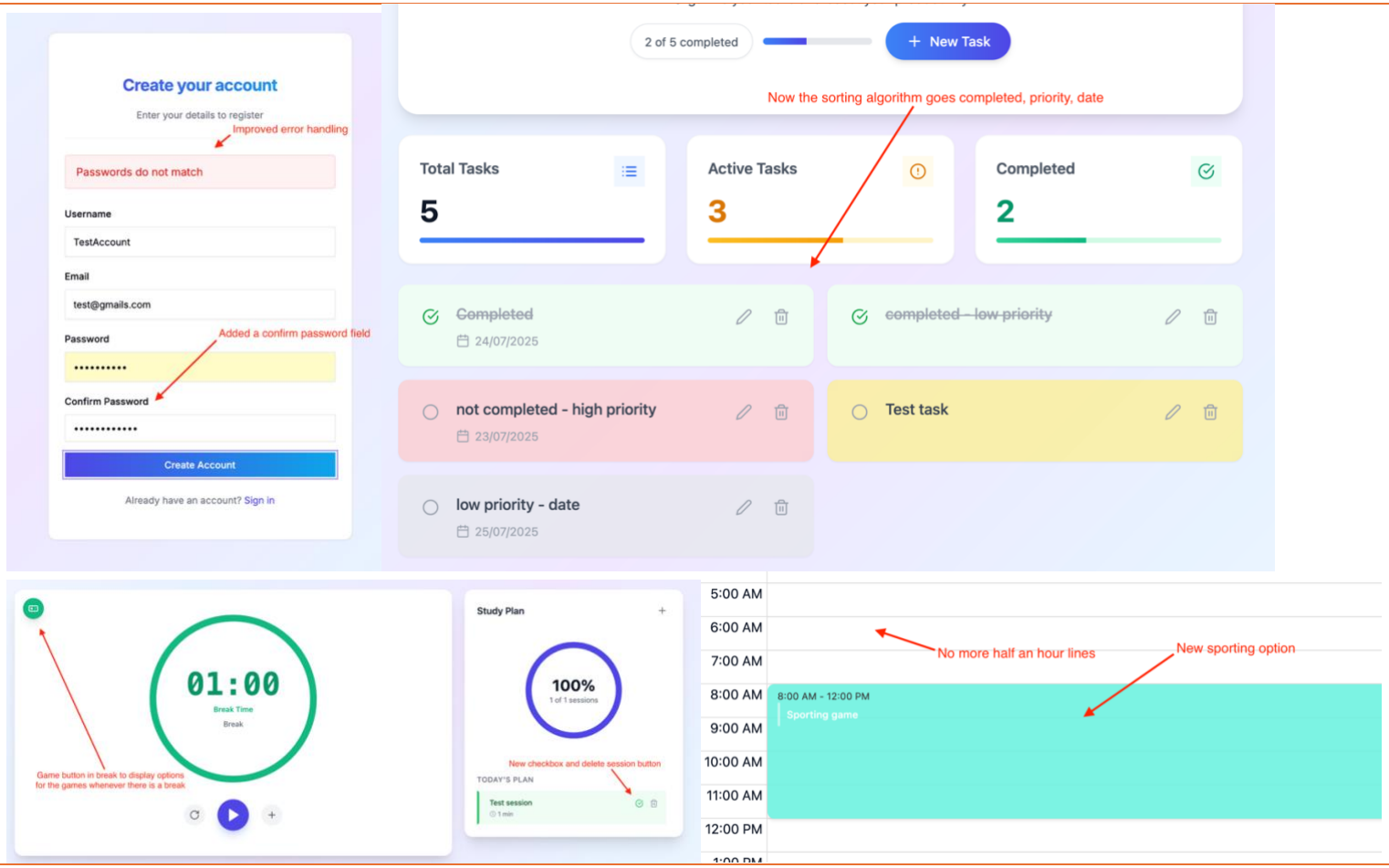
Date:	2/7/25
Title:	Study Planner, study games
Author:	Jake Handel
Tasks Description:	Created the study planner and study games alongside it to make the timer functional.
Image:	
Logbook Entry:	Progress since the last entry: Created the study timer page. Popups to control the flow of information. Minesweeper, Tetris, snake games added in. Task Achieved: Completed the study planner and the games. Stumbling blocks or issues encountered and how they were managed: Making sure there was a correct flow, so, it went from the creation, to start, then complete, then break options, and continue branching off that to make

	sure it flowed correctly and the right points. Also, the backend was being difficult for some reason in saving the sessions. Possible approaches for upcoming tasks: Next task is adding in the settings and linking to dashboard. So correct database usage, and use “dark:” tailwind features. Reflective comments: Study planner took a lot of time and debugging, still a few things needing to be tidied but can be fixed easily.
Resources:	Pomodoro technique.

Date:	4/7/25
Title:	Settings and dark mode
Author:	Jake Handel
Tasks Description:	Small settings popup and integrating dark mode.
Image:	
Logbook Entry:	Progress since the last entry: Settings popup on sidebar and a dark mode feature. Task Achieved: Detailed dark mode. Stumbling blocks or issues encountered and how they were managed: Just making sure styles were consistent and utilised proper formatting. Making sure the dark mode didn't show immediately after creating an account. Possible approaches for upcoming tasks: Just bug fixes. Get users to test everything and identify any bugs. Reflective comments: Looking great, settings are very responsive, and dark mode is great for my eyes to have a break from the light.

Resources:	N/A

Date:	6/7/25
Title:	Dashboard syncing and limiting console logs
Author:	Jake Handel
Tasks Description:	Dashboard has stats on the study timer and limited to no console logging in the terminal now.
Image:	
Logbook Entry:	Progress since the last entry: No console logs (removing debugging lines), and dashboard now receives stats from the study timer. Task Achieved: Dashboard complete. This is version 1 now, I am now able to get feedback on it, to improve but the main Campus Koala is done. Stumbling blocks or issues encountered and how they were managed: Making sure the right stats were being formed, and it was consistent styling. Possible approaches for upcoming tasks: Mainly finding bugs and getting feedback now. Reflective comments: Happy that it is mostly done now, a few minor details to fix up but otherwise it is solid.
Resources:	N/A

Date:	21/7/2025
Title:	Feedback and Pseudocode adjustments
Author:	Jake Handel
Tasks Description:	Implementing some feedback changes I got from users and changing some of the Pseudocode algorithms in the VS editor.
Image:	

Logbook Entry:	Progress since the last entry: Fixing the feedback given from user surveys. Moving and commenting on the pseudocode algorithms (see images above for some of the major fixes. Task Achieved: Fixed majority of the feedback that was given. Stumbling blocks or issues encountered and how they were managed: No stumbling blocks or issues occurred. Possible approaches for upcoming tasks: Future tasks include further versions and new elements to the app for commercial use. Reflective comments: Webapp has come out well and works.
Resources:	N/A

Milestone 3.1: This will be checked at various points each term.

Section 4 - Testing and Maintaining (10 Marks)

4.1 — Acceptance Testing

Apply methodologies to test and evaluate whether the application meets all outlined requirements and objectives. Describe the acceptance testing process, including test cases, scenarios, and criteria. Document the results and any necessary actions.

The environment:

- I am testing it locally on a MacBook M3 Pro on Sequoia 15.5 with 18GB of ram
- Using the terminal to host the frontend and the backend using npm run dev and python3 app.py
- Using Firefox developer edition with over 500mb of memory left
- WIFI: downloading speed of 47.8mb/s, uploading speed of 16.7mb/s.
- The website loads efficiently and works fluently with the backend and frontend. It has a smooth connection and processing speed.

Objective	How was it tested	Did it meet expectations	Any actions needed
Login/ Registering	While developing I tested the logic through manually entering data into the registering system and then checking how it works in the login. I used console logging and debugging statements to see how the data was flowing to fix any issues. Later I used Apache JMeter to input any invalid and security threats to test the	Yes, it met the expectations that it needed to. It did its job in allowing a user to sign up and login to their account, storing their information correctly and securely. It also gives them a token at login for extra security.	A user suggested that the registering had an easier password follow, so if they were missing a capital letter it would tell them instead of a generic message, which I have implemented. This objective is solid, so I would say no actions are needed.

Objective	How was it tested	Did it meet expectations	Any actions needed
	responses, and it worked well with the correct responses. I used user feedback to fix and adjust any minor feedback they suggested.		
Todo list tasks	The Todo list tasks were tested by manually inputting valid and invalid data points to test the websites response. Users could enter the information they needed, and it would display in a correct format. I then used JMeter to input security measures to test its response to different inputs without website validation and it handled the responses well.	The Todo list hits expectations with its intended purpose and is easy to use UI. It allowed for people to edit, create and delete tasks as needed. It kept users organised so they could keep on track with their tasks.	No actions needed for functionality; feedback was more just optional and user suggestions. I had a suggestion to sort the tasks differently so that all the completed tasks were at the top of the list instead of jumbled around all the priorities, so I fixed that. A user suggested that in the dashboard they could click on a recent task, and it would take them to that task, so it has been fixed.
Calendar events	The calendar events were first tested by manually inputting different fields and testing the response and handling of the website. I had different users use the calendar to how they would want to use it to test	Yes mostly, I was able to create, edit and delete events, while maintaining a good UI and easy to use GUI. However, the edit functionality is a little bugged in the fact that it changes the time and date each time you update it, so it has not met its expectation on that. For the deleting and	I need to fix the time and date of the calendar updating so that it doesn't update every time. More of how it looks, and the functionality make a day scroll if it has more events on during the day, so it is easier to read. A user suggested to change the default time from 12AM to 8AM as it is more of a default time and a regular time. I have implemented

Objective	How was it tested	Did it meet expectations	Any actions needed
	how well it achieved its objectives of creating, editing and deleting events. I then used JMeter again to pentest and security test the inputs and the handling, and it worked well with the inputs I gave it.	creation of the event it has met the requirements.	that change now. A user also wants a sports section in the calendar which I will be implementing.
Syncing Todo list tasks with calendar	I tested this by manually putting a date onto tasks and then checking if they have synced across to the calendar. I then tested whether it would change the event if I changed the task in the Todo list, which it did.	It's simple objective of getting tasks with a date to the calendar was achieved. It displays tasks with a date in the calendar under its category "Todo", which is what I needed it to do.	Nothing to fix as it syncs nicely with the calendar page and allows me to edit and the task as I need to and it syncs up with each other. So, no further actions needed to be taken.
Day planner	The day planner was quite difficult to test given the nature of having to wait for the timer to finish to continue testing the flow of the timer, and how it completed and flowed the data. It was tested through manual data entry as entering data through JMeter would not have tested the flow of the timer and just the input of data, so the manual data allowed me to test numerous key points about the data and the flow of the information being passed through each stage of the timer.	The day planner was supposed to be a place where a user could plan their session for the time they wanted and then the following sessions. I merged it into study sessions as it worked similarly but was more just a different timed study session. This objective was partially met as users can plan of what they would like to complete; however, they can't plan for the time at which everything starts.	There are a few issues with this that I have received as feedback and through testing. After each session the study sessions delete themselves so you cannot track the progress, so I added this feedback in the feedback branch, and you can delete it manually is needed. Another minor issue that was suggested was the plus button is there for just presentation when there are no tasks, but users would like it to have functionality by being able to add sessions and plan their sessions out. Overall, need to improve on a few minor issues with the day planner but otherwise the core functionality is there.

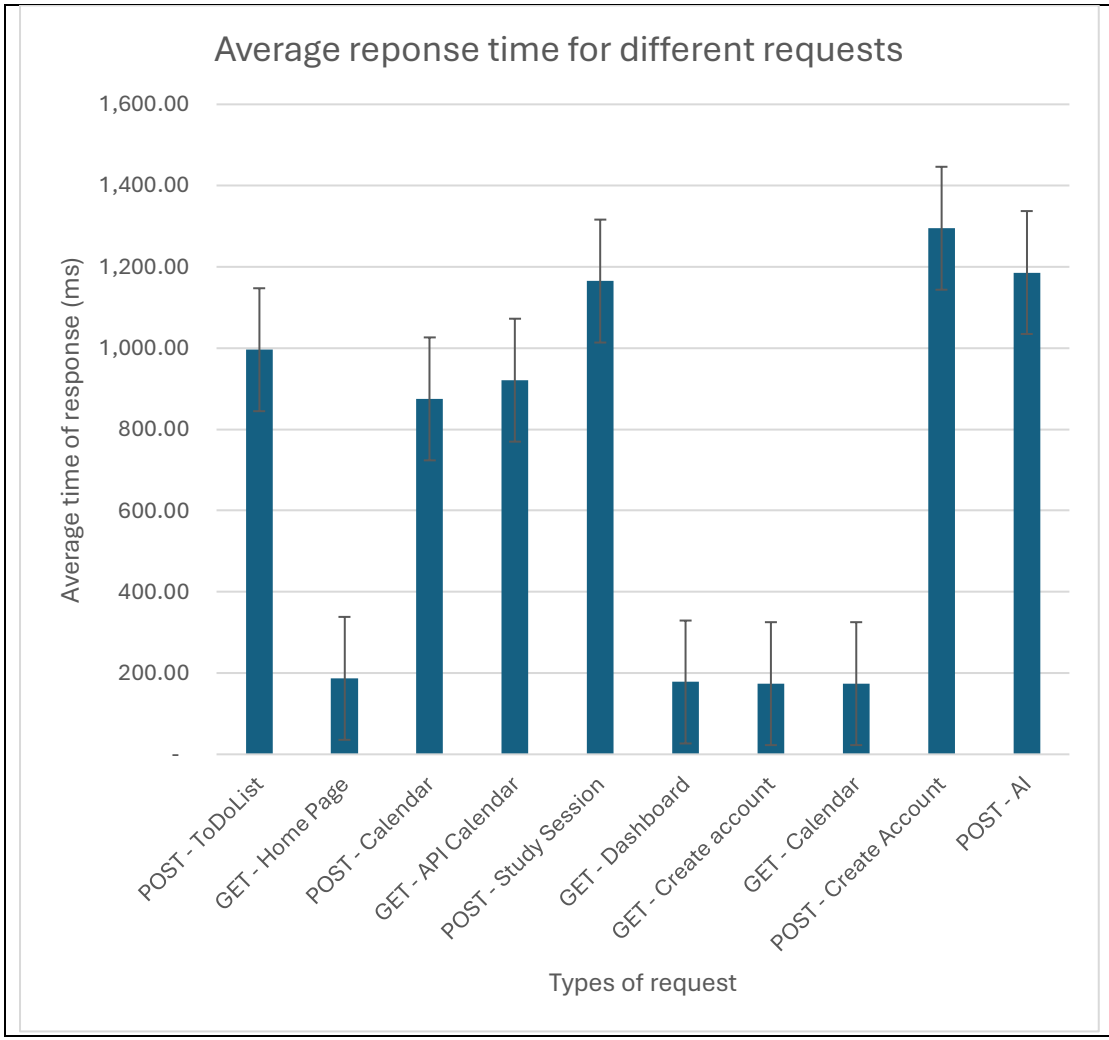
Objective	How was it tested	Did it meet expectations	Any actions needed
	It was also tested by different users testing the app to see how they would use it and find any issues with the timer itself.		
Study timers	The study timer component was tested through manual data entry as a lot of the testing done to it was about the flow of data and how it acted when the time was up. Hence, I tested this through manual data entry and many timers.	Its objective was to be able to have a simple 25-minute session as a default or start a customised time. It achieved this, users can either start a free study session for 25 mins or start a planned session up to 120 mins. At the conclusion it will congratulate the user and bring up the break options (discussed below).	This timer works quite well at its basic point of starting and completing it time. However, users have found that when planning multiple sessions, they cannot change to a different time and it will get stuck on a set time, unable to change unless finished. So, this still needs work. Otherwise, the basic functionality is achieved.
Break timers	The break timer was tested through manual data entry and user experience as JMeter is unable to test the flow. I tested using many data sets repeatedly to test how the breaks flowed and the different options for the break.	Its objective was to start a break timer equal to 1/3 of the study timer. This has been achieved. Users have the choice of either to playing a game, free break or continue to study. Once they have clicked an option, the break timer will then begin for 1/3 of the time, as it was supposed to. At the conclusion it will display what they would like to do after the break, study or have another break. Making the objective complete and meet expectations.	This works very well and needs limited work and allows users to do what they need. There is a minor bug in that if you press another break or press play to start a break, it will not display any options for the break so that still needs fixing but everything else is functional.
Games	The games were tested by people as they needed to be tested by users that would like to play the games. So, by	The objective was to create engaging games. I built three working and functional games: Tetris, Snake and Minesweeper. All	The games themselves are functional and work well. However, the high scores are not saved, and I am undecided if I wanted to add

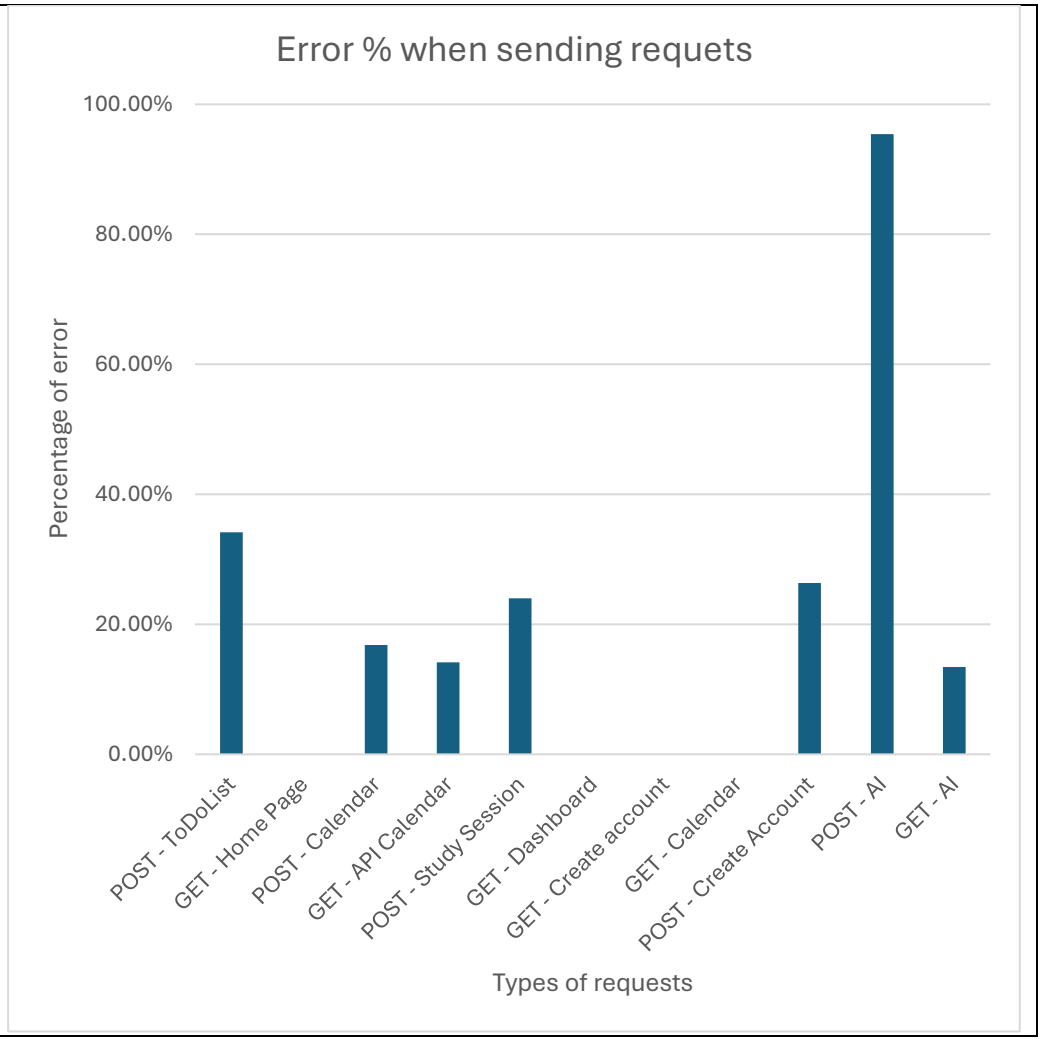
Objective	How was it tested	Did it meet expectations	Any actions needed
	allowing myself to have a long study session I was able to test games out and let others play the game, which helped debug my Snake, Minesweeper and Tetris games, and improve them. They were mostly tested by users playing each one for a minute and trying to find any bugs in it.	hit their objective and work. As it is a break, the game automatically gets finished once the timer runs out to keep the user on track with their study. This also helps improve their study and focus. So, the games are good in restricting how long they break for and their mental focus.	that in as a feature, as the objective is to create a study break and not a competitive scored environment. Another minor bug is that once you are in the game you cannot go back to choose another game; you are only able to go to the study timer screen and continue as a free study time. This has now been fixed through a button in the break menu that is always displayed so you can get back to the games.
AI Assistant	The AI assistant was tested through load testing and manual testing to see the limits of the AI and the responses. Through the load testing I tested vulnerabilities such as SQL injection and XSS attacks and was unsuccessful. In the manual testing, I was trying to break the APIs response and was able to restrict the output of the AI to make sure it wasn't returning uninterpretable AI responses and was readable for users.	The objective of the user being able to ask a question and receive a response was achieved. Further, the user has a recollection of saved conversations, and the AI has memory to recall what has been said in previous chats. The user can start and go back to different conversations allowing for them to rename their conversations to what they need.	There aren't any major actions needed to be taken. Maybe only to fix a few UI changes so that a user once they submit a request it displays a spinning popup to show the user it is thinking. Further making it resizable will allow the user to view it more easily. So, not big changes but minor enhancements.
Dark Mode	Dark mode was tested through manual testing and checking every page it was applied to. Then also reloading and new account defaults.	Its objective was to be able to switch from light and dark mode for a user's preference and it achieved it. It uses the users operating systems preference for dark or light mode and automatically changes their profile to their preference, but users can change it in their settings page, back and forth from light and dark mode.	No changes or additions need to be added or changes to the dark mode feature as it works well and smoothly. It is liked by users as well due to the clean UI in both the light and dark mode.

4.2 — Load Testing

Apply methodologies to test and evaluate the application's stability and performance under different levels of usage and load. Detail the load testing procedures, tools, and metrics. Provide a report on the findings and recommendations for improvement.

Label	# Samples	Average (ms)	Min (ms)	Max (ms)	Std. Dev.	Error %	Throughput (# of requests/ time taken)	Received KB/sec	Sent KB/sec	Avg. Bytes
POST - Todo List	500.00	996.00	-	6,200.00	1,176.37	34.20%	26.99	30.78	12.49	1,167.80
GET - Home Page	500.00	187.00	29.00	567.00	95.95	0.00%	59.00	1,664.07	27.94	28,883.00
POST - Calendar	500.00	875.00	-	8,148.00	1,028.95	16.80%	22.67	20.44	15.42	923.30
GET CALENDAR EVENTS	500.00	921.00	-	10,272.00	1,122.91	14.20%	22.62	4,200.35	9.44	190,154.30
POST - Study Session	500.00	1,165.00	-	7,724.00	1,245.47	24.00%	24.76	22.67	12.04	937.60
GET - Dashboard	500.00	178.00	40.00	532.00	86.65	0.00%	58.80	1,638.19	28.36	28,531.00
GET - Create account	500.00	174.00	35.00	497.00	85.53	0.00%	59.40	500.35	28.60	8,625.00
GET - Calendar	500.00	174.00	20.00	588.00	92.92	0.00%	59.52	560.47	28.65	9,643.00
POST - Create Account	500.00	1,295.00	-	8,369.00	1,256.78	26.40%	24.59	28.53	12.23	1,188.10
POST - AI	500.00	1,186.00	-	10,850.00	1,226.55	95.40%	21.70	14.45	10.19	681.70
GET - AI	500.00	734.00	-	5,334.00	926.87	13.40%	22.24	909.15	9.57	41,865.70


Average reponse time for different requests
1,600.00
1,400.00
1,200.00
1,000.00
800.00
600.00
400.00
200.00
-
Average time of response (ms)
POST - ToDoList
GET - Home Page
POST - Calendar
GET - API Calendar
POST - Study Session
GET - Dashboard
GET - Create account
GET - Calendar
POST - Create Account
POST - AI
Types of request


Error % when sending requets
100.00%
80.00%
60.00%
40.00%
20.00%
0.00%
Percentage of error
POST - ToDoList
GET - Home Page
POST - Calendar
GET - API Calendar
POST - Study Session
GET - Dashboard
GET - Create account
GET - Calendar
POST - Create Account
POST - AI
GET - AI
Types of requests

Load Testing Summary Table

Category	Details
Testing Tool	Apache JMeter
Test Setup	50 users sending 10 requests each
Metrics Evaluated	- Response time (ms) - Error rate (%) - Throughput (data in/out)
Key JMeter Metrics Explained	- # Samples: Number of requests sent - Average/Min/Max: Response time in ms - Error %: Failure rate - Throughput: Data sent/received

Response Time Findings

Request Type / Route	Average Response Time	Notes
Static 'GET' Requests (Webpage)	Very Low	Quick loading and stable under load; no improvements needed
API Requests (General)	1024ms	Backend and frontend interaction causes higher latency
Create Account	1300ms	Backend handling: user validation + SQL database setup
AI Prompt Submission	1186ms	External API call to Gemini-1.5-Flash with significant processing
Create Study Timer	1165ms	Anomalous — other creations avg 935ms; possibly due to inefficient code
Other Create Requests (e.g., calendar/to-do)	935ms	Within expected range and reasonable

Performance Considerations

Category	Observations
Stability	Static GET requests show strong stability under high load
Variation in API Routes	Standard deviation of /api routes: **1140ms** — shows inconsistency
Improvement Recommendations	- Optimise backend routing - Streamline code for the study timer creation logic

Error Rate Findings

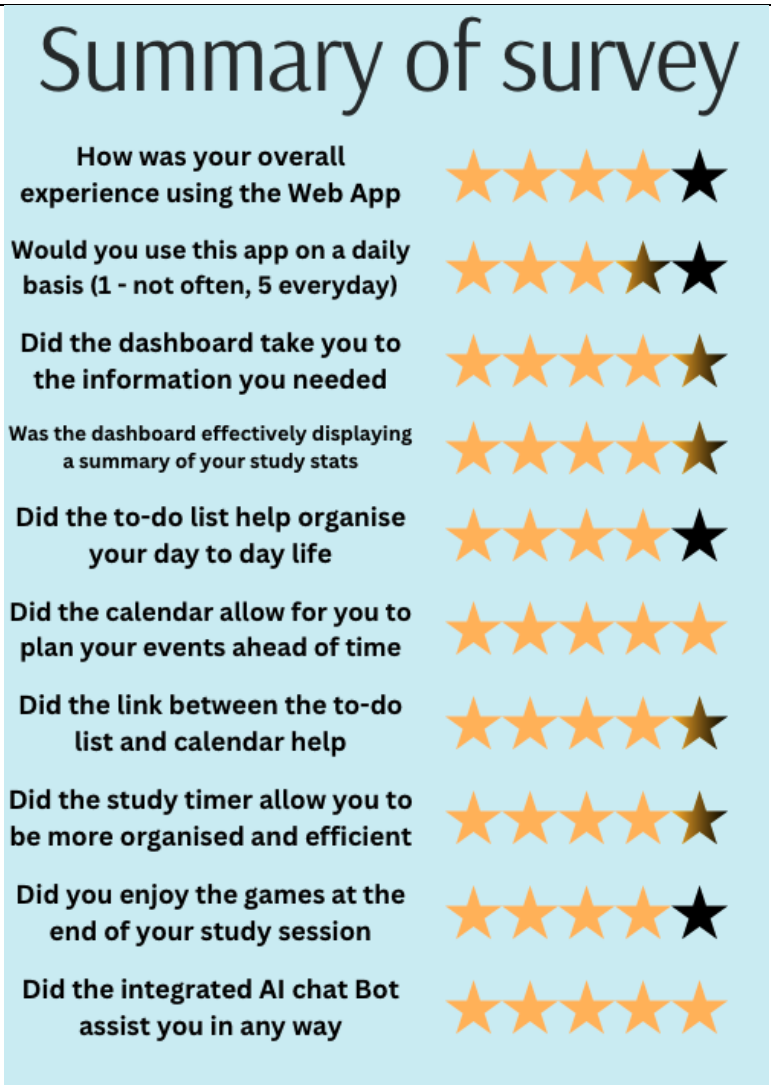
Cause	Details
AI Prompt Submission	- High error rate due to Gemini-1.5-Flash free tier - Limited to 15 requests/min & 50/day (source) - Upgrade to paid tier or alternate model needed
Create User / Task	- Higher error rates due to strict duplicate prevention - Double ups caused by rapid sequential handling
AI Conversation Fetching	- Occasional 404 errors - Caused by automation not retrieving correct AI conversation - Not user-accessible; no fix required

Overall Summary & Recommendations

Area	Evaluation
Response Time	– Generally good, some variability acceptable given backend processing
Errors	– Mostly explainable by usage limits or duplicate handling; not critical except AI API limit
Recommendations	– Optimise inefficient API endpoints – Consider upgrading AI API plan Minor backend refactoring for consistency – Ongoing review and enhancement of code to drive efficiency and user improvements and removal of bugs

4.3 — User Survey

Collect feedback through a survey to analyse and respond to feedback on the application's usability, functionality, and overall satisfaction. Design a user survey with relevant questions and distribute it to a representative sample of users. Summarise the survey results and outline the changes made based on user feedback.



I surveyed approximately 15 year 12 students from different schools. The survey used (as noted above) included these 10 questions and a rating out of 5. The questions were reflective of the main goals for the webapp.

The above results demonstrate a high level of success, and the overall goals of the application were achieved.

In addition to the rating questions, I asked users detailed written questions including:

- what was the most useful
- what would you change
- is there something you would add; and
- was there something you didn't find useful?

Many people said that they found the to-do list and calendar component works well together to keep them on track, "*I found the calendar integration stuff the most useful because it allowed me to view my events that were upcoming*".

Another key feedback was how useful the quick links and the display of information was in the dashboard, allowing for summarised versions to show their progress. The feedback was very useful in finding minor bugs and feedback. Based on the feedback received, the following was provided which were then built into the updated version of the application:

Feedback response	Addressed or Future Version
The calendar time updates when updating an event	Addressed
When there is more than one study timer planned, you are unable to switch between them	Future version
Easier sign up with password	Addressed
Need to go back to the games while playing the games	Addressed
Get rid of half an hour line in calenda	Addressed
Minesweeper start may only give one square	Addressed
Study planner sessions don't deletion	Addressed
Pop up calendar error details	Addressed
Study timer time duration input	Addressed
Plus, button needs to add sessions	Addressed
Break session after break needs games option	Addressed
Set calendar default time 8am	Addressed
Todo list completed to the top	Addressed
Page not found Task in dashboard	Addressed
View all events not clickable	Addressed
Calendar Sport section	Addressed
AI Error codes	Addressed
Check password on signup	Addressed
Study timer needs to pop more	Future version
Back to dashboard	Future version
Background animations	Future version

Through this feedback, I was able to refine and adjust my webapp to what users wanted and needed. After receiving feedback, I was able to complete majority of these issues (as seen by the strikethroughs).

As for the future of this project users recommended that I add:

- Calendar and Todo list notifications for reminders
- Customisable widgets for the dashboard
- Study music
- AI chat bot that accepts and produces visuals
- Integration with iPhone apps for easy access.

Overall users liked the aesthetic nature of the webapp and the ease of use, while finding the to-do list and the calendar useful to organise and schedule their plans well in advance. Users were of the view that the application met the outcomes set out but more importantly is a tool that they would use as it meets what they see as a need to help them with their study activities.